

第 14 章: Iterations and Comprehensions (迭代与推导式)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本章是第 13 章循环语句的直接延续, 系统讲解 Python 的迭代协议 (iteration protocol) 和推导式 (comprehensions) 两大主题。这是全书的分水岭: 搞懂 "for 循环背后到底发生了什么", 后面第 20 章的生成器 (generators)、第 30 章的类级可迭代对象才能顺理成章。读完本章, 你就掌握了 Python 中"一切从左到右扫描的工具"的共同底盘。

14.1 开篇引言 (Chapter Introduction)

英文原文

In the prior chapter, we met Python's two looping statements, while and for. Although they can handle most repetitive tasks programs need to perform, iterating over collections is so common and pervasive that Python provides additional tools to make it simpler and more efficient. This chapter begins our exploration of these tools. Specifically, it presents Python's iterati

on protocol, a method-call model used by the for loop, and fills in some details on comprehensions, which are a close cousin to the for loop that apply an expression to each item in a collection. Because these tools are related to both the for loop and functions, we'll take a two-pass approach to covering them in this book—along with a postscript: This chapter introduces their basics in the context of looping-based tools, serving as something of a continuation of the prior chapter. Chapter 20 revisits them in the context of function-based tools, and extends the topic to include built-in and user-defined generators. Chapter 30 provides a shorter final installment in this story, which will show us how to code user-defined iterable objects with classes. One note up front: some of the concepts presented in these chapters may seem advanced at first glance. With practice, though, you'll find that these tools are useful and natural. Although never strictly required, they've also become commonplace in Python code, so a basic understanding can help if you must read programs written by others.

中文翻译

在上一章中，我们认识了 Python 的两条循环语句：while 和 for。虽然它们能处理程序需要执行的大多数重复性任务，但遍历集合（iterating over collections）是如此常见和普遍，以至于 Python 提供了额外的工具来让这件事更简单、更高效。本章开始探索这些工具。具体来说，本章呈现 P

python 的迭代协议 (iteration protocol) ——一种被 for 循环使用的方法调用模型——并补充一些关于推导式 (comprehensions) 的细节；推导式是 for 循环的近亲，它把某个表达式应用到集合中的每一个元素上。因为这些工具既与 for 循环有关，又与函数 (functions) 有关，本书将用“两遍走”的方式来覆盖它们——外加一个后记：本章在基于循环的工具语境中介绍它们的基础，算是上一章某种意义上的延续。第 20 章会在基于函数的工具语境中重新讨论它们，并把话题扩展到内建的和用户自定义的生成器 (generators)。第 30 章则提供这个故事更短的最后集，它将展示如何用类 (classes) 编写用户自定义的可迭代对象。先说一句：这些章节中呈现的某些概念，乍一看可能显得很高级。但通过练习你会发现，这些工具既实用又自然。虽然从来不是严格必需的，它们已成为 Python 代码中的常见写法，所以如果你必须阅读别人写的程序，基本的了解会很有帮助。

深度理解

·核心概念：本章回答一个根本问题——for 循环为什么能“通吃”一切（列表、元组、字符串、字典、文件、range）？答案是迭代协议：Python 所有从左到右扫描的工具都建立在同一套方法调用模型上。理解“一个协议、无数工具”是本章的钥匙。

·为什么用“两遍走”：Lutz 特意把迭代和推导式拆成三章（14、20、30）讲，是因为这些工具横跨语句 (looping) 和函数 (functions) 两大阵营，还与类 (classes) 相关。先借语句语境建立直觉，再借函数语境补全细节，最后

用类完成收尾——这是"螺旋式教学"的典型设计。

·生活例子：迭代协议就像插座标准。任何电器（文件、字典、range）只要按标准做插头（实现 iter/next），就能插入任何插座（for、推导式、map、zip）。本书三章就是在逐步教你"插头的制作工艺"。

·初学者误区：误以为 for 循环是 Python 内部写死的魔法。实际上 for 只是"协议消费者"，任何对象都能实现协议成为被迭代者——这正是第 30 章的核心。

·实际问题：为什么这些工具"常见于他人代码"？因为 Python 社区崇尚"迭代式处理"（惰性、省内存、表达力强），读不懂迭代协议，读别人的 Python 代码会处处卡壳。

14.2 迭代 (Iterations)

英文原文

In the preceding chapter, we learned that the for loop can work on any sequence type in Python, including lists, tuples, and strings, like this (with blank lines required by REPLs after compound statements omitted for brevity):

```
>>> for x in [1, 2, 3, 4]: print(x 2, end='') 1 4 9 16
>>> for x in (1, 2, 3, 4): print(x 3, end='') 1 8 27 64
>>> for x in 'text': print(x 2, end='') tt ee xx tt
```

As we've also learned, the for loop is even more generic than this—it works on any iterable object. In fact,

this is true of all iteration tools that scan objects from left to right in Python, including for loops; comprehensions of all stripes; some in membership tests; the zip, enumerate, and map built-in functions; and more. Any iterable object will do, even nonsequences like dictionaries:

```
>>> for k in dict(a=1, b=2, c=3): print(k, end=" ") a b c
```

The concept of iterable objects was added to Python after its inception, but it has come to permeate the language's toolset. It's essentially a generalization of the notion of sequences—an object is considered iterable if it is either a physically stored sequence or an object that produces one result at a time in the context of an iteration tool like for. In a sense, iterable objects include both real sequences and virtual sequences computed on demand. The virtual sequences both save memory and avoid delays by producing results one at a time, instead of all at once. These are not true sequences, however: virtual iterables do not support the full range of operations defined for lists and tuples. Rather, they simply materialize a series of values over time and on request. Whether an iterable is physical or virtual, it announces its support for iterations by implementing the iteration protocol—a set of callable methods used by all iteration tools, and the subject of the next section. > NOTE: Terminology moment (术语时刻) : The Python world sometimes uses the

terms "iterable" and "iterator" interchangeably (and confusingly!) to refer to an object that supports iteration in general. For clarity, this book uses the term `iterable` to refer to an object that has the `iter` call at the top of the protocol we're about to meet, and `iterator` to refer to an object that has the `next` call to produce results. That is, an `iterable` returns an `iterator` that advances on `next`. This book also uses the phrase `iteration tool` for language tools that run an iteration, like `for` loops and `zip` calls. Chapter 20 will muddle this jargon with the term `generator`—which refers to objects that automatically support the iteration protocol, and hence are `iterable`—even though all `iterables` generate results!

中文翻译

在上一章中，我们学到 `for` 循环可以作用于 Python 中任何序列（sequence）类型，包括列表、元组和字符串，就像这样（为了简洁，这里省略了 REPL 在复合语句后要求输入的空白行）：

```
>>> for x in [1, 2, 3, 4]: print(x**2, end=" ")
1 4 9 16 >>> for x in (1, 2, 3, 4): print(x**3, end=" ")
1 8 27 64 >>> for x in 'text': print(x**2, end=" ")
t t e e x x t t
```

我们还学到，`for` 循环比这更通用——它作用于任何可迭代（iterable）对象。事实上，Python 中所有从左到右扫描对象的迭代工具（iteration tools）都是如此，包括 `for` 循环；各式各样的推导式；部分 `in` 成员测试；`zip`、`enumerate` 和 `map` 内建函数；以及更多。任何可迭代对象都可以，即使是像字典这样的非序列对象：

```
>>> for k in dict(a=
```

1, b=2, c=3): print(k, end='') a b c 可迭代对象这个概念是在 Python 诞生之后才加入的，但它已经渗透到这门语言的整个工具集。它本质上是"序列"概念的推广——如果一个对象是物理存储的序列，或者它能在像 for 这样的迭代工具的语境中一次产生一个结果，那么这个对象就被认为是可迭代的。从某种意义上说，可迭代对象既包括真正的序列，也包括按需计算的虚拟 (virtual) 序列。虚拟序列一次只产生一个结果，而不是一次性全部产出，从而既节省内存又避免延迟。然而，它们不是真正的序列：虚拟可迭代对象不支持为列表和元组定义的完整操作集。相反，它们只是在时间推移中、按请求逐步"物化" (materialize) 出一系列值。无论一个可迭代对象是物理的还是虚拟的，它都通过实现迭代协议 (iteration protocol) 来宣告自己对迭代的支持——迭代协议是一组可调用的方法，被所有迭代工具使用，也是下一节的主题。> 注意：术语时刻：Python 世界有时会把 "iterable" (可迭代对象) 和 "iterator" (迭代器) 这两个词混用 (而且很令人困惑!)，都用来泛指支持迭代的对象。为了清晰起见，本书用可迭代对象 (iterable) 指代拥有我们即将认识的协议顶端 iter 调用的对象，用迭代器 (iterator) 指代拥有用于产生结果的 next 调用的对象。也就是说，可迭代对象返回一个在 next 时前进的迭代器。本书还用迭代工具 (iteration tool) 这个短语指代"运行一次迭代"的语言工具，比如 for 循环和 zip 调用。第 20 章还会用生成器 (generator) 这个术语把这套行话搅浑——生成器指自动支持迭代协议、因此是可迭代的对象——尽管所有可迭代对象都会"生成"结果!

深度理解

·核心概念：iterable（可迭代对象）是"序列"概念的一般化。物理序列（列表、元组、字符串）是"存好了等着你读"，虚拟序列（文件、range、zip 结果）是"你问一次、我给一个"。两者共同点：都能一次提供一个元素，供迭代工具消费。

·底层实现：本章节先展示"现象"（for 能遍历一切），下一节揭示"机制"（iter/next/StopIteration）。注意 dict(a=1, b=2, c=3) 的遍历只给出键 a b c——字典的迭代器默认产生键，这是 Python 的设计决定。

·为什么这样设计：Python 1990 年诞生时还没有可迭代对象的概念（那时只有序列）；1998 年 Python 2.2 引入迭代协议（PEP 234）。为什么后来要加？因为"一次一个"地产生结果能同时省内存、省时间，且让文件、网络流这类"没有长度、无法下标"的对象也能优雅地参与循环。

·生活例子：物理序列像整箱矿泉水（搬过来才能开箱拿），虚拟序列像饮水机（要一杯接一杯）。饮水机的好处：不用囤货、随取随有；代价：不能"直接拿第 100 杯"（不支持下标 [99]）。

·初学者误区：① 把 iterable 和 iterator 混为一谈——本书明确约定：iterable 是"提供者"（有 iter），iterator 是"推进器"（有 next），前者返回后者；② 以为字典 for k in D 遍历的是"键值对"——默认是键；③ 以为所有可迭代对象都支持 len、下标——虚拟序列不支持，这正是"非序列"的含义。

·实际问题：理解虚拟序列后，"文件怎么逐行读""range 为什么不直接给列表""zip 的结果为什么是 zip object"这些困惑全部迎刃而解。

14.3 迭代协议 (The Iteration Protocol)

英文原文

One of the easiest ways to understand the iteration protocol is to see how it works with a built-in type such as the file object we first explored in Chapter 9. In this chapter, we'll be using the following three-line input file as a demo:

```
>> print(open('data.txt').read()) Testing file IO Learning Python, 6E Python 3.12 >> open('data.txt').read()
Testing file IO\nLearning Python, 6E\nPython 3.12\n'
```

Recall from Chapter 9 that open file objects have a method called `readline`, which reads one line of text from a file at a time—each time we call the `readline` method, we advance to the next line. At the end of the file, an empty string is returned, which we can detect to break out of a line-reading loop (remember, empty means false):

```
>> f = open('data.txt')

# Read a three-line file in this directory >> f.readline()

```

```
# readline loads one line on each call
Testing file IO
>> f.readline()
```

```
# Newlines are \n everywhere in text mode
Learning Python, 6E
>> f.readline()
```

```
# Last lines may have a \n or not
Python 3.12
>> f.readline()
```

```
# Returns empty string at end-of-file"
```

Files, however, also have a method named `next` that has a nearly identical effect—it returns the next line from a file each time it is called. The only noticeable difference is that `next` raises a built-in `StopIteration` exception (that is, invokes a signaling event) at end-of-file instead of returning an empty string:

```
>> f = open('data.txt')
```

```
# next loads one line on each call too
>> f.next()
```

```
# But raises an exception at end-of-file
Testing file IO
>> f.next()
'Learning Python, 6E'
>> f.next()
'Python 3.12'
>> f.next()
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
StopIteration
```

And this interface is most of what we call the iteration protocol in Python. Any object with a `next` method to advance to a next result, which raises `StopIteration` at the end of the series of results, is considered an iterator in Python.

Any such object may also be stepped through with a

for loop or any other iteration tool, because all iteration tools normally work internally by calling next on each iteration and catching the StopIteration exception to know when to exit. As you'll see in a moment, for some objects the full protocol includes an additional first step to call iter, but this isn't required for files.

The upshot of all this magic is that, as mentioned in Chapters 9 and 13, the generally best way to read a text file line by line today is to not read it at all—instead, allow the for loop to automatically call next to advance to the next line on each iteration. The file object's iterator will do the work of automatically loading lines as you go, both one at a time and efficiently.

The following, for example, reads line by line, printing the uppercase version of each line along the way—without ever explicitly reading from the file at all:

```
>> for line in open('data.txt'):
# Use file iterators to read by lines print(line.upper(),
end='') # Calls next, catches StopIteration TESTING FI
LE IO LEARNING PYTHON, 6E PYTHON 3.12
```

Notice that the print uses end="" here to suppress adding a \n, because line strings already have one (without this, our output would be double-spaced). This for coding pattern is usually the best way to read text files line by line, for three reasons: it's the simplest to code, might be the quickest to run, and uses memory space sparingly.

Prior to the advent of the iteration protocol in Python, programmers achieved the same effect with a for loop by calling the file readlines method to load the file's content into memory as a list of line strings:

```
>> for line in open('data.txt').readlines():  
    print(line.upper(), end='') TESTING FILE IO LEARNING  
PYTHON, 6E PYTHON 3.12
```

This readlines technique still works, but is not considered the best practice today because it performs poorly in terms of memory usage. In fact, because this version really does load the entire file into memory all at once, it will not even work for files too big to fit into the memory space available on your device.

By contrast, the iterator-based version is immune to such memory-explosion issues because it reads just one line at a time. The iterator version might run quicker too, though this can vary per Python release (but see the upcoming note for a few specs).

As mentioned in the prior chapter's closing sidebar, "Why You Will Care: File Scanners", it's also possible to read a file line by line with a while loop:

```
>> f = open('data.txt')  
>> while line:= f.readline():  
    print(line.upper(), end='') TESTING FILE IO LEARNING  
PYTHON, 6E PYTHON 3.12
```

However, this may run slower than the iterator-based for loop version because file iterators run at C-lang

uage speed inside the standard CPython, whereas the while version must run Python bytecode through the Python virtual machine. Anytime we trade Python code for C code, speed tends to increase.

This is not an absolute truth, though; again, we'll explore timing techniques in Chapter 21 for measuring the relative speed of alternatives like these, though the following note ruins some of the surprise for the impatient.

> NOTE: Spoiler alert (剧透警告) : Per calls to `min(itertools.repeat(code, repeat=50, number=10))` in CPython 3.12 on macOS, the file iterator is still slightly faster than `readlines`, which is faster than the while loop. With a 9k-line file and this chapter's code (using `pass` for loop bodies), the iterator, `readlines`, and while alternatives check in at 0.0073, 0.0077, and 0.0102 seconds, respectively.

The while is slowest and using `:=` doesn't help much (it's 0.0104 sans `:=`). For more info, see the examples package and Chapter 21. Caveats: your test variables may vary, memory matters too, and 0.0029 seconds may not be enough to get excited about in some programs.

中文翻译

理解迭代协议最容易的方法之一，是看看它如何与文件（file）对象这样的内建类型协作——我们在第 9 章首次接触过

文件。本章将使用下面这个三行的输入文件作为演示：>>> print(open('data.txt').read()) Testing file IO Learning Python, 6E Python 3.12 >>> open('data.txt').read() Testing file IO\nLearning Python, 6E\nPython 3.12\n回忆第 9 章：打开的文件对象有一个叫 readline 的方法，它一次从文件读取一行文本——每次调用 readline 方法，就前进到下一行。在文件末尾，它返回一个空字符串，我们可以检测到这个空串从而跳出读行循环（记住，空意味着假）：>>> f = open('data.txt') # 读取本目录下的三行文件>>> f.readline() # readline 每次调用加载一行'Testing file IO\n'>>> f.readline() # 文本模式下换行符处处都是 \n'Learning Python, 6E\n'>>> f.readline() # 最后一行可能有 \n 也可能没有'Python 3.12\n'>>> f.readline() # 文件末尾返回空字符串"然而，文件还有一个名叫 next 的方法，效果几乎完全相同——每次调用它，它就从文件返回下一行。唯一明显的区别是：next 在文件末尾抛出内建的 StopIteration 异常（也就是说，触发一个"信号事件"），而不是返回空字符串：>>> f = open('data.txt') # next 每次调用同样加载一行>>> f.next() # 但在文件末尾会抛出异常'Testing file IO\n'>>> f.next()'Learning Python, 6E\n'>>> f.next()'Python 3.12\n'>>> f.next() Traceback (most recent call last): File "<stdin>", line 1, in <module> StopIteration 而这个接口，就是我们在 Python 中所称迭代协议（iteration protocol）的大部分内容。任何带有 next 方法（用于前进到下一个结果）、并在结果序列末尾抛出 StopIteration 的对象，在 Python 中都被视为迭代器（iterator）。任何这样的对象都可以被 for 循环或任何其他迭代工具逐步遍历，因为所有迭代工具在内部通常

都是这样工作的：每次迭代调用 `next`，并捕获 `StopIteration` 异常来得知何时退出。稍后你会看到，对某些对象来说，完整的协议还包含一个额外的第一步：调用 `iter`；但对文件来说这一步不是必需的。这一切魔法的最终结论是，正如第 9 章和第 13 章提到的：如今逐行读取文本文件的最佳方式，是根本不去“显式地读”——而是让 `for` 循环在每次迭代时自动调用 `next` 前进到下一行。文件对象的迭代器会在你迭代的过程中自动加载每一行，既一次一行，又高效。例如下面这段代码就逐行读取，并沿途打印每行的大写版本——全程从未显式读取文件：

```
>>> for line in open('data.txt'): # 用文件迭代器按行读取
    print(line.upper(), end='')
# 调用 next, 捕获 StopIteration
TESTING FILE IO LEARNING PYTHON, 6E PYTHON 3.12
注意这里的 print 使用了 end="" 来抑制添加 \n, 因为每行字符串已经自带一个换行符了 (不加的话, 我们的输出就会隔行打印)。这种 for 编码模式通常是逐行读取文本文件的最佳方式, 原因有三: 写起来最简单、运行起来可能最快、内存占用最节省。在 Python 引入迭代协议之前, 程序员用 for 循环实现同样效果的方法是调用文件的 readlines 方法, 把整个文件内容作为列表 (list) 载入内存:
```

```
>>> for line in open('data.txt').readlines():
    print(line.upper(), end='')
TESTING FILE IO LEARNING PYTHON, 6E PYTHON 3.12
```

这种 `readlines` 技巧仍然有效，但在今天已不算最佳实践，因为它的内存表现很差。事实上，由于这个版本真的会把整个文件一次性加载进内存，对于大到超出你设备可用内存的文件，它根本无法工作。相比之下，基于迭代器的版本不会出现这种“内存爆炸”问题，因为它一次只读一行。迭代器版本可能运行得也更快，尽管这因 Python 版本而异（详见后面的注）。正

如上一章结尾的侧栏"你会关心什么：文件扫描器" (Why You Will Care: File Scanners) 提到的，用 while 循环也能逐行读文件：

```
>>> f = open('data.txt')
>>> while line := f.readline(): print(line.upper(), end="")
```

TESTING FILE IO LEARNING PYTHON, 6E PYTHON 3.12 不过，它可能比基于迭代器的 for 循环版本更慢，因为文件迭代器在标准 CPython 内部以 C 语言的速度运行，而 while 版本必须让 Python 字节码 (bytecode) 穿过 Python 虚拟机 (virtual machine) 运行。任何时候我们用 C 代码替换 Python 代码，速度往往会提升。但这也不是绝对真理；我们会在第 21 章探索计时技术，来衡量这类替代方案之间的相对速度——不过下面这个注提前剧透了部分悬念，性子急的读者不必等。

> 注意：剧透警告：根据 CPython 3.12 在 macOS 上的 `min(timeit.repeat(code, repeat=50, number=10))` 调用测试，文件迭代器仍比 `readlines` 略快，而 `readlines` 又比 while 循环快。对一份 9000 行的文件、使用本章的代码（循环体用 `pass`），迭代器、`readlines`、while 三种方案的耗时分别为 0.0073、0.0077 和 0.0102 秒。while 最慢，用 `:=` 也没多大帮助（不用 `:=` 是 0.0104）。更多信息见示例包和第 21 章。注意事项：你的测试变量可能不同，内存也很重要，而且 0.0029 秒的差距在有些程序里可能根本不值得激动。

深度理解

·核心概念：迭代协议的"骨架"是 `next + StopIteration`。协议的本质是约定："下一个结果"由 `next` 给，"没有结果了"由 `StopIteration` 异常来宣告。异常在这里不是"出错"，而是"信号事件" (signaling event) ——一种优雅的终

止通知。

·底层实现：for line in f 展开后大致等价于：内部循环反复调用 f.next()，直到捕获到 StopIteration 才跳出。CPython 中文件迭代器的 next 是在 C 层实现的，每次调用只把缓冲区里的一行数据返回，几乎零额外开销；而 while line := f.readline() 每轮循环都要解释执行 readline 调用、海象赋值、比较判断等一串字节码，所以更慢。这就是“用 C 换掉 Python 代码，速度就上去了”的机制根源。

·为什么这样设计：为什么用“空字符串”之外的方式（异常）来标记文件结尾？因为空字符串可能是合法内容（空行），用哨兵值有歧义；而异常传播路径是专线，永远不会和正常数据混淆。这正是“让协议可组合”的关键——任何迭代工具只需捕获 StopIteration，无需理解每个对象的具体终止规则。

·解决什么实际问题：逐行处理大文件（日志、CSV、基因组数据）。readlines 把整个文件塞进内存，10GB 的日志直接内存爆炸；迭代器方案内存占用恒等于一行的大小。

·初学者误区：① 以为 readline 和 next 是一回事——区别在结尾行为（空串 vs 异常），理解不了这一点就看不懂 for 为何能自动停止；② 写文件循环时手动 while True: line = f.readline(); if not line: break——用海象表达式虽然能简化，但最地道的写法就是让 for 代劳；③ 误以为 print(line.upper(), end='') 中的 end="" 是“去掉换行”——实际上是“不追加换行”，因为行本身已含 \n。

代码分析

```
>>> f = open('data.txt')           #
>>> f.readline()                   # readline
'Testing file IO\n'
>>> f.readline()                   # \n
'Learning Python, 6E\n'
>>> f.readline()                   # \n
'Python 3.12\n'
>>> f.readline()                   #
''
```

解释器如何处理：open('data.txt') 在 CPython 内部创建 io.TextIOWrapper 对象，它带有一个 C 层实现的缓冲区。readline() 在 C 层扫描缓冲区，找到第一个 \n 为止，把这一段拷贝成新的 Python 字符串对象返回，同时把文件位置指针前移；到文件末尾时，由于没有更多数据，返回空字符串"（此时内部会自动尝试向操作系统读取更多数据，读到 EOF 就放弃）。注意：readline() 与迭代协议没有关系——它是文件的普通方法。

```
>>> f = open('data.txt')           # __next__
>>> f.__next__()                   #
'Testing file IO\n'
>>> f.__next__()
'Learning Python, 6E\n'
>>> f.__next__()
'Python 3.12\n'
>>> f.__next__()                   #
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

解释器如何处理：f.next() 走的是迭代协议通道。CPython 中 TextIOWrapper 的 next 在 C 层复用与 readline 相

同的行读取逻辑，区别只在到达文件末尾时的行为：不再返回"，而是抛出一个 `StopIteration` 类型的实例。Trace back 显示这个异常从 C 层一路传播到 REPL 顶层，因为没有 for 循环或任何迭代工具替我们捕获它。这正是 for 循环在内部悄悄做的事——所以这个异常对我们"透明"，只有手动调用 `next` 时才会看见它。

```
>>> for line in open('data.txt'): #  
    print(line.upper(), end='') # __next__ StopIteration  
TESTING FILE IO  
LEARNING PYTHON, 6E  
PYTHON 3.12
```

解释器如何处理：① 求值 `open('data.txt')` 得到文件对象；② for 循环通过内部等价于 `iter()` 的机制取得迭代器（文件返回它自己）；③ 每轮循环：调用文件对象的 `next()` 取下一行，若抛出 `StopIteration` 就正常退出循环；④ 把取得的行绑定给变量 `line`，执行循环体。整个过程全部在解释器内部完成，无任何 Python 级字节码开销在"取下一行"这件事上——这就是它比 while 版本快的原因。`print(line.upper(), end='')` 里 `str.upper()` 先创建新字符串对象，`print` 再以 `end=""` 抑制追加换行。

设计思想：for 循环的优雅之处在于——它把"如何取数据"（协议细节）和"如何处理数据"（循环体）彻底解耦。迭代器负责"每次给一个"，for 负责"给到何时为止"。这把三行样板代码（读行、判空、break）压缩为一行循环头，也让"逐行读文件"这种操作在任何可迭代对象上都成立。

14.4 iter 与 next 内建函数 (The iter and next Built-ins)

英文原文

To simplify manual iteration code, Python also provides a built-in function, `next`, that has the same net effect as calling an object's `next` method. That is, given an iterator object `X`, the call `next(X)` is the same as `X.next()`, but is noticeably simpler to type and read (and actually runs slightly faster in CPython 3.12 for tested cases). With files, for instance, either form may be used:

```
>> f = open('data.txt') >> f.next()

# Call iteration method directly
Testing file IO\n>> next(f)

# next(f) is the same as f.next()
Learning Python, 6E\n'>> next(f)
Python 3.12\n'>> next(f)

... exception text omitted from here on ...
StopIteration
```

Technically, there is one more piece to the iteration protocol alluded to earlier. When the `for` loop begins, it first obtains an iterator from an iterable object, by calling the iterable's `iter` method. The object returned by this call in turn has the required `next` method to advance. For convenience again, the `iter` built-in function

on internally runs the equivalent of the iter method, much as next runs the equivalent of next.

Hence, for loops run the internal equivalent of the following, though the iter step is moot and optional for files—they are their own iterators, because files don't support multiple scans per open:

```
>> f = open('data.txt') >> I = iter(f)
# Fetch an iterator from an iterable >> next(I)
# Fetch the next result from the iterator'Testing file I
O\n'>> next(I)
# Files iterables are iterators themselves'Learning Python, 6E\n'
... etc...
```

中文翻译

为了简化手动迭代代码，Python 还提供了一个内建函数 `next`，其净效果与调用对象的 `next` 方法相同。也就是说，给定迭代器对象 `X`，调用 `next(X)` 等同于 `X.next()`，但明显更容易输入、更容易阅读（在 CPython 3.12 的测试用例中实际上运行还略快一些）。比如对文件，两种形式都可以用：

```
>>> f = open('data.txt') >>> f.next() # 直接调用迭代方法'Testing file IO\n'>>> next(f) # next(f) 与 f.next() 等价'Learning Python, 6E\n'>>> next(f)'Python 3.12\n'>>> next(f) ...异常文本从这里起省略... StopIteration
```

严格来说，之前提到的迭代协议还有一块拼图。当 `for` 循环

开始时，它首先通过调用可迭代对象的 `iter` 方法，从一个可迭代对象 (iterable) 那里获得一个迭代器 (iterator)。这次调用返回的对象依次拥有前进所需的 `next` 方法。为了方便起见，`iter` 内建函数在内部运行的是 `iter` 方法的等价物，就像 `next` 运行 `next` 的等价物一样。因此，`for` 循环在内部运行的等价代码如下——尽管对文件来说，`iter` 这一步是无意义且可选的：文件本身就是自己的迭代器，因为每次 `open` 的文件不支持多次扫描：

```
>>> f = open('data.txt')
>>> I = iter(f) # 从可迭代对象获取一个迭代器
>>> next(I) # 从迭代器获取下一个结果'Testing file IO\n'
>>> next(I) # 文件的可迭代对象本身就是迭代器'Learning Python, 6E\n'...等等 ...
```

深度理解

- 核心概念：`iter(X)` 与 `next(X)` 是两个"门面"内建函数：`iter` 是 `iter` 的便捷入口，`next` 是 `next` 的便捷入口。它们不改变协议本身，只是把双下划线方法包装成好读、好打的普通函数。

- 底层实现：CPython 中 `next(f)` 直接调用 `f` 的 `tpiternext` 槽位函数 (C 层)，而 `f.next()` 需要先完成一次"属性查找→绑定方法→调用"的过程，所以 `next()` 略快——不过差异是微秒级的。`iter(f)` 调用 `tpiter` 槽位；对文件来说该槽位返回对象自身，所以 `iter(f)` 是 `f`。

- 为什么这样设计：Python 的惯例是"双下划线方法留给语言机制，普通函数留给人类" (如 `len/len`、`iter/iter`、`next/next`)。双下划线 (dunder) 方法可能被任何对象重载，而内建函数层可以附带额外的参数与容错 (比如 `next`

(X, default)) , 为语言未来的演进留出空间。

·解决什么实际问题：需要手动推进迭代器时（比如交替消费两个迭代器、实现"预取下一项"的 lookahead 逻辑），写 `next(it)` 比 `it.next()` 清晰得多。

·初学者误区：① 记不清 `iter/next` 谁配 `iter`、谁配 `next`——助记："迭代器"取迭代器，`iter(iterable) → iterator`；`next` 就是"下一个"。② 误以为 `iter` 只对序列有效——任何可迭代对象都行；③ 忘记 `for` 循环本身会调用 `iter`，手动代码里写 `for x in iter(L)` 纯属多此一举（无害但冗余）。

14.5 完整的迭代协议 (The Full Iteration Protocol)

英文原文

With all these pieces in place, Figure 14-1 sketches the full iteration protocol, used by every iteration tool in Python and supported by a wide variety of object types. It's based on two objects used in two distinct steps by iteration tools: - The iterable object for which iteration is requested.

Calling this object's `iter` returns an iterator, and is the same as calling `iter`. - The iterator object returned by the iterable. Calling this object's `next` produces results during the iteration and raises `StopIteration` when no more results remain, and is the same as calling `next`

Figure 14-1. The iteration protocol, used by for loops, comprehensions, maps, and more. These steps are orchestrated automatically by iteration tools in most cases, but it helps to understand these two objects' roles.

For example, in some cases these two objects are the same when only a single scan is supported (e.g., files), and the iterator object is often a temporary, used internally by the iteration tool. Moreover, some objects are both an iteration tool (they iterate) and an iterable object (their results are iterable)—including the enumerate and zip built-ins, and Chapter 20's generator expressions.

Such iterables already avoid constructing result lists in memory themselves, but applying them to other iterables saves even more space. When such tools are combined, no work is done until an iteration tool requests results.

In code, the protocol's first step becomes obvious if we look at how for loops internally process built-in sequence types such as lists (for uses the internal equivalents of the "" methods, but you use either in your code):

```
>> L = [1, 2] >> I = iter(L)
```

Obtain an iterator object from an iterable >> next(I)

)

Call next(iterator) to advance to next item 1 >> next(l)

Or use L.iter() and l.next() calls 2 >> next(l) StopIteration

As we saw earlier, the initial iter step is not required for files, because a file object supports just one scan and hence is its own iterator. You can see this yourself with is (recall from Chapter 9 that this means object identity—the same exact piece of object memory, not just the same value):

```
>> f = open('data.txt') >> iter(f) is f
```

```
# Files are iterators themselves: iter() optional True >
> iter(f) is f.iter()
```

```
# Both calls return file object f itself True >> next(f)
```

```
# Which responds to next requests directly'Testing file IO\n'
```

Lists and many other built-in objects, though, are not their own iterators because they do support multiple open iterations—there may be any number of iterations active in nested loops, and all may be at different positions at a given point in time. For such objects, we must call iter to start iterating manually:

```
>> L = [1, 2, 3] >> iter(L) is L
```

```
# Lists are not their own iterators: use iter() False >>
next(L) TypeError:'list' object is not an iterator >> l =
iter(L)

# Same as L.iter() >> next(l)

# Same as l.next() 1
```

中文翻译

有了所有这些拼图，图 14-1 勾勒出了这个完整的迭代协议——它被 Python 中每一个迭代工具使用，并被形形色色的对象类型支持。它基于两个对象，由迭代工具分两个不同的步骤使用：- 可迭代对象 (iterable)：为其请求迭代的对象。调用该对象的 `iter` 返回一个迭代器，等价于调用 `iter`。- 迭代器 (iterator)：由可迭代对象返回的对象。调用该对象的 `next` 在迭代过程中产生结果，在没有更多结果时抛出 `StopIteration`，等价于调用 `next`。图 14-1. 迭代协议：被 `for` 循环、推导式、`map` 以及更多工具使用在大多数情况下，这些步骤由迭代工具自动编排，但理解这两个对象的角色仍有帮助。例如，在只支持单次扫描的情况下（如文件），这两个对象有时是同一个；而迭代器对象往往是临时对象，由迭代工具在内部使用。此外，有些对象既是迭代工具（它们会迭代）又是可迭代对象（它们的结果可迭代）——包括 `enumerate` 和 `zip` 内建函数，以及第 20 章的生成器表达式。这类可迭代对象自身已经避免在内存中构造结果列表，而把它们应用到其他可迭代对象上还能省下更多空间。当这些工具组合在一起时，在迭代工具请求结果之前，什么工作都不会做。在代码层面，只要看看 `for` 循环在内部

如何处理列表这样的内建序列类型，协议的第一步就一目了然了（for 使用"方法的内部等价物，而你在代码里用哪种写法都行）：

```
>>> L = [1, 2] >>> I = iter(L) # 从可迭代对象获取迭代器对象
>>> next(I) # 调用 next(iterator) 前进到下一项1
>>> next(I) # 或者用 L.iter() 和 I.next() 调用2
>>> next(I) StopIteration
```

正如我们前面看到的，对文件来说最初的 iter 步骤不是必需的，因为文件对象只支持一次扫描，因此它本身就是自己的迭代器。你可以用 is 亲自验证这一点（回忆第 9 章：is 表示对象同一性（identity）——是同一块对象内存，而不只是值相同）：

```
>>> f = open('data.txt')
>>> iter(f) is f # 文件本身就是迭代器：iter() 可省略True
>>> iter(f) is f.iter() # 两次调用都返回文件对象 f 本身True
>>> next(f) # 它直接响应 next 请求'Testing file IO\n'
```

不过，列表和许多其他内建对象并不是自己的迭代器，因为它们确实支持多个同时进行的迭代（multiple open iterations）——嵌套循环中可以有任意数量的迭代同时活跃，且在同一时刻它们可能各自处于不同位置。对于这类对象，手动开始迭代时必须调用 iter：

```
>>> L = [1, 2, 3]
>>> iter(L) is L # 列表不是自己的迭代器：要用 iter() False
>>> next(L) TypeError:'list' object is not an iterator
>>> I = iter(L) # 与 L.iter() 等价
>>> next(I) # 与 I.next() 等价1
```

深度理解

·核心概念：完整的迭代协议是“两对象、两步走”：iterable.iter() 产生 iterator，iterator.next() 产生结果，StopIteration 宣告终止。这四样东西（两方法、两对象）就是 Python 迭代世界的全部基石。

·底层实现：CPython 中 for 循环编译成 FORITER 字节码，它会调用迭代器的 tpiternext 槽位；若返回 NULL 且设置了 StopIteration 异常，解释器就吞掉异常、正常退出循环。对列表：iter(L) 会创建新的 listiterator 对象，内部持有指向列表的指针和独立的位置索引——所以两个迭代器互不干扰。对文件：tpiter 直接返回自身，位置状态就存在文件对象里，天然只能扫一遍。

·为什么这样设计：把"可迭代对象"和"迭代器"分成两个角色，是为了支持多次独立扫描：列表可以被多个嵌套循环同时遍历，每个迭代器记住自己的位置。若迭代器就是对象本身（如文件），则"位置"只有一份，只能单遍。这是"职责分离"在语言协议中的体现。

·生活例子：可迭代对象像一张 CD（内容在盘上），迭代器像 CD 播放器（带磁头位置）。一张 CD 可以同时接多个播放器，各自从头播、位置互不影响（iter(L) 两次得到两个播放器）；而文件更像磁带机——机器和磁带是一体的，一盘磁带同时只能有一个磁头，读到哪就是哪，倒带（重新 open）才能重来。

·初学者误区：① 在列表上直接调 next(L) 报 TypeError: 'list' object is not an iterator——列表是 iterable 不是 iterator，必须先 iter(L)；② 用 iter(f) is f 判断文件"是它自己的迭代器"，注意 is 是身份比较（同一块内存），不是值比较 ==；③ 以为所有对象都能像列表一样重复遍历——文件、zip、enumerate 结果用一次就耗尽。

·实际问题：这个区别直接决定代码怎么写——需要多次扫描（例如先统计行数再处理）就必须用列表或 readlines

把数据"固化"下来；单遍处理则放心用迭代器，省内存。

代码分析

```
>>> L = [1, 2]
>>> I = iter(L)           #
>>> next(I)              # next(iterator)
1
>>> next(I)              # L.__iter__() I.__next__()
2
>>> next(I)
StopIteration
```

解释器如何处理：iter(L) 在 C 层调用 list 类型的 tpiter 槽位，构造一个 listiterator 对象：它保存两个 C 字段——指向列表的 itseq 引用和整数索引 itindex。每次 next(I) 触发 tpiternext：若 itindex < len(L)，取出 L[itindex]、索引加 1、返回该元素；索引越界时，释放对列表的引用 (itseq = NULL) 并抛出 StopIteration。整个过程没有调用任何 Python 级方法，所以手动迭代的开销极小。

```
>>> f = open('data.txt')
>>> iter(f) is f          # iter()
True
>>> iter(f) is f.__iter__() # f
True
>>> next(f)              # next
'Testing file IO\n'
```

解释器如何处理：io.TextIOWrapper 的 tpiter 返回 self（引用计数加 1），所以 iter(f) is f 为 True。is 比较的是 PyObject 指针地址，这里两边是同一个文件对象。文件把"当前位置"（一个缓冲区指针加偏移量）存在自身，这意味着不存在第二个独立位置——这是它不能多次扫描的

根因。相比之下，列表能返回任意多个 listiterator，每个都带独立索引，所以 iter(L) is L 为 False。

设计思想：协议对"单遍"与"多遍"一视同仁——迭代工具只认协议，不关心背后是文件还是列表。这种"面向协议而非面向实现"的设计，使第 30 章里用户自定义的可迭代类能无缝接入所有迭代工具。

14.6 手动迭代 (Manual Iteration)

英文原文

Although Python iteration tools call these functions automatically, we can also use them to apply the iteration protocol manually when needed. The following demonstrates the equivalence between automatic and manual iteration (again, for runs the internal equivalent of `l.next` instead of the `next(l)` used here, but the effect is the same):

```
>>> L = [1, 2, 3]
>>> for X in L:
# Automatic iteration
print(X 2, end="")
# Obtain iterator, call next, catch exceptions
1 4 9
>>> l = iter(L)
# Manual iteration: what for loops usually do
>>> while True:
# Use try statements to catch stop exception
try: X = next(l)
except StopIteration: break
print(X 2, end="")
1 4 9
```

To understand this code, you need to know that try statements run an action and catch exceptions that occur while the action runs (we met exceptions briefly in Chapter 10 but will explore them in depth in Part VII).

中文翻译

虽然 Python 的迭代工具会自动调用这些函数，但在需要时，我们也可以用手动方式应用迭代协议。下面这段代码演示了自动迭代与手动迭代之间的等价关系（再次说明，for 运行的是 `l.next` 的内部等价物，而不是这里使用的 `next(l)`，但效果相同）：

```
>>> L = [1, 2, 3]
>>> for X in L:
# 自动迭代
    print(X ** 2, end=" ")
# 获取 iter、调用 next、捕获异常
1 4 9
>>> l = iter(L)
# 手动迭代：for 循环通常做的事
>>> while True:
# 用 try 语句捕获停止异常
    try:
        X = next(l)
    except StopIteration:
        break
    print(X ** 2, end=" ")
1 4 9
```

要理解这段代码，你需要知道：try 语句执行一个动作，并捕获该动作运行过程中出现的异常（我们在第 10 章简单接触过异常，但会在第七部分深入探讨它们）。

深度理解

·核心概念：for 循环不是魔法，它是“迭代协议 + 异常处理”的语法糖。手动迭代就是把这层糖剥开：`iter()` 取迭代器 → 循环里 `next()` 取元素 → `except StopIteration` 退出。看懂这段等价代码，才算真正看懂 for。

·底层实现：for 循环编译后的 FORITER 字节码与手动 try/except 的差异在于：FORITER 在解释器内部捕获 StopIteration 时不会展开完整的异常处理路径（没有堆栈回溯、没有异常对象流转），而是直接跳转到循环结束指令，因此比手动 try/except 更轻快。这也是为什么“让 for 代劳”通常更快。

·为什么这样设计：把终止信号设计成异常，是为了让协议

可以穿透任意层级的嵌套代码——`next` 可以在任何深度被调用，`StopIteration` 都能准确传到捕获点，无需每层函数显式传递"结束标志"。

·生活例子：手动迭代像手工点钞：你（`while`）一张张数（`next`），数完了（`StopIteration`）就停（`break`）；自动迭代像点钞机：你把钱放进去（`for`），机器自己数完自己停。两者结果一样，但机器（解释器内部）更快、更省心。

·初学者误区：① 忘了 `iter(L)` 这一步，直接 `next(L)` 报 `TypeError`；② 在 `except` 分支里忘记 `break`，导致 `StopIteration` 被吞掉后无限循环；③ 不知道 `next` 抛出异常时循环变量可能保持旧值——手动迭代时若需"最后有效值"要自己保存。

·实际问题：手动迭代的唯一真实用途是在无法用 `for` 表达的推进方式里，比如"隔一个取一个"、"先取开头再处理剩余"，以及下一节讲的 `next(X, default)` 与海象表达式的组合。

14.7 关于 `iter` 与 `next` 的更多内容 (More on `iter` and `next`)

英文原文

For full fidelity, it should also be noted that for loops and other iteration tools can sometimes work differently for user-defined classes—repeatedly indexing an

object instead of running the iteration protocol—but they prefer the iteration protocol when supported (more on this story when we study operator overloading in Chapter 30). Though not commonly used, it's also worth noting that `next` accepts an optional second default argument for an exit value; if passed, it's returned at the end instead of raising a stop exception:

```
>>> L = [1]
>>> I = iter(L) # Result instead of exception
>>> next(I, 'end of list')
1
>>> next(I, 'end of list')
'end of list'
```

Combined with the `:=` named-assignment expression, this can shave multiple lines off the preceding manual-iteration code—but will also fail if the passed default can appear as a valid result:

```
>>> I = iter(L)
>>> while (X := next(I, None)) != None: # Same effect, less code
    print(X * 2, end='') # Assuming None is safe!
```

Though also uncommon, `iter` accepts a second sentinel argument to signal stop from a callable. This very different mode provides an arguably tricky way to use `for` to read files by blocks—but requires info on functions or `lambda`, which we don't yet have:

```
>>> f = open('data.txt')
>>> I = iter(lambda: f.read(5), '') # Callables and sentinels
>>> for block in I:
    print(block, end='') # Assuming you know lambda!
```

Watch for `lambda` in the next part of this book, and see Python's docs for more on `next` and `iter` modes.

中文翻译

为了完整起见，还应该指出：for 循环和其他迭代工具有时对用户自定义类的工作方式不同——反复对对象做下标索引（indexing），而不是运行迭代协议——但在协议可用时，它们会优先使用协议（当我们研究第 30 章的操作符重载（operator overloading）时再细说这个故事）。虽然不常用，但值得注意：next 接受一个可选的第二个默认值（default）参数，作为退出值；如果传入，迭代结束时返回它，而不是抛出停止异常：

```
>>> L = [1] >>> I = iter(L)
# 返回结果而不是异常>>> next(I,'end of list') 1 >>>
next(I,'end of list')'end of list'
```

与 := 命名赋值表达式（named-assignment expression）结合使用，它可以从前面的手动迭代代码中省掉多行——但如果传入的默认值可能作为合法结果出现，这种方法就会失效：

```
>>> I = iter(L)
>>> while (X := next(I, None)) != None: # 效果相同，
# 代码更少print(X 2, end='') # 假设 None 是安全的！ 同样
# 不常用的还有：iter 接受第二个哨兵（sentinel）参数，用于
# 让一个可调用对象（callable）发出停止信号。这种非常
# 不同的模式提供了一种颇有争议的技巧，可用 for 按块读取
# 文件——但它需要函数或 lambda 的知识，我们目前还没有：
```

```
>>> f = open('data.txt') >>> I = iter(lambda: f.read(5), "")
# 可调用对象与哨兵值>>> for block in I: print
# (block, end='') # 假设你懂 lambda！ 留意本书下一部分
# 中的 lambda，更多关于 next 和 iter 模式的说明参见 Python
# 文档。
```

深度理解

·核心概念：next 和 iter 各有一个“隐藏的第二个参数”：next(X, default) 用默认值替代终止异常；iter(callable, se

ntinel) 反复调用 callable 直到返回值等于 sentinel。它们让"手动迭代"和"按块读取"这两种特殊需求有了优雅的单行解。

·底层实现：next(X, default) 在 C 层捕获 StopIteration 后不重新抛出，而是返回 default。iter(func, sentinel) 创建 callableiterator：每次 next 调用 func()，若结果与 sentinel 相同（用 == 比较）就抛 StopIteration，否则返回该结果——文件按块读就是每 5 个字符一个结果，读到"（EOF）停止。

·为什么这样设计：这两个模式是"协议之上的协议"。next 的默认值参数让"取完即止"的逻辑无需写异常处理；哨兵模式把"重复调用直到某条件"（典型的 C 风格循环）压进迭代器，从而复用 for 的全部表达能力。

·生活例子：next(l, default) 像"自动售货机缺货时吐出一张'售罄'纸条"而不是报警；iter(func, sentinel) 像自动点钞机的"验钞模式"——机器一张张验（调用 func），验到假币（sentinel）就停机。

·初学者误区：① 用 while (X := next(l, None)) != None 时如果数据里真出现 None，循环会提前终止——注释里的"假设 None 是安全的！"就是在提醒这个坑，Lutz 自己也承认这是"tricky"（刁钻）写法；② 把哨兵模式与普通模式混用——iter(f)（单参数）和 iter(func, sentinel)（双参数）语义完全不同；③ 以为 next 的默认值参数会在异常时"静默"——是的，但任何其他异常（如 TypeError）仍然照常抛出。

·实际问题：哨兵模式在读取二进制块、轮询设备状态、读 `readline` 的变体等"重复调用直到哨兵"场景中极其实用，也是很多框架内部实现"读取直到结束"的惯用手法。

14.8 其他内建可迭代对象 (Other Built-in Iterables)

英文原文

Besides files and physical sequences like lists, many other objects in Python have useful iterators as well. Now that we have a better handle on how the iteration protocol works, this section revisits some tools we've already seen in this context, and introduces a handful of additional iterables along the way.

中文翻译

除了文件和列表这样的物理序列，Python 中还有许多其他对象也带有实用的迭代器。既然我们现在对迭代协议的工作方式有了更好的把握，本节就从这个角度回顾一些我们已经见过的工具，并顺带介绍一批额外的可迭代对象。

深度理解

- 核心概念：迭代协议不是文件的专利——字典、`range`、`enumerate`、`zip`、`map`、`filter`、标准库工具全都实现它。学会一个协议，等于学会所有迭代工具的打开方式。
- 底层的统一性：所有这些对象都在 C 层实现了 `tpiter/tpi`

ternext 槽位，所以 for、list()、推导式对它们一视同仁。差异只在"是否支持多次扫描"。

·为什么这样设计：协议统一了"怎么取值"，把精力留给"取什么值"的业务差异，这正是语言工具集可以无限扩充而不增加学习成本的原因。

14.8.1 回顾：字典、range、enumerate 与 zip (Reprise: Dictionaries, range, enumerate, and zip)

英文原文

As we saw in the last chapter, the usual way to step through the keys of a dictionary is with a for loop:

```
>> D = dict(a=1, b=2) >> for key in D:
```

```
# Dictionaries are implicitly iterable print(key, D[key])
a 1 b 2
```

This works simply because dictionaries are iterables with an iterator that automatically returns one key at a time in an iteration tool like for:

```
>> I = iter(D)
```

```
# Which just means they support the protocol >> next(I)'a'>> next(I)'b'>> next(I) StopIteration
```

The iteration protocol is also the reason that we've had to wrap range results in a list call to see their values all at once in a REPL. Objects that are nonsequenc

e iterables return results one at a time, not in a physical list:

```
>> R = range(5) >> R range(0, 5) >> I = iter(R)
# Use iteration protocol to produce results >> next(I)
0 >> next(I) 1 >> list(range(5))
# Or use list() to collect all results at once [0, 1, 2, 3, 4]
]
```

Note that the list call here is not needed and should n't be used in contexts where iteration happens automatically—such as within for loops. It is needed for displaying values all at once, though, and may also be required when list-like behavior or multiple scans are required for objects that normally produce results on demand (more on this later).

Now that you have a better understanding of this protocol, you should also be able to see how it explains why the enumerate tool introduced in the prior chapter works the way it does:

```
>> E = enumerate('text')
# enumerate is an iterable too >> E <enumerate object at 0x1010ab880> >> I = iter(E) >> next(I)
# Generate results with iteration protocol (0,'t') >> next(I)
# Or use list() to force generation to run (1,'e') >> list(enumerate('text')) [(0,'t'), (1,'e'), (2,'x'), (3,'t')]
```

Unlike range, the enumerate built-in's result is its own iterator, much like files. This means it supports just one scan per call, and iter is optional (more on this a head too):

```
>> R = range(5) >> iter(R) is R False >> E = enumerate('text') >> iter(E) is E True >> next(E) (0,'t')
```

The zip built-in, covered in the prior chapter, works the same way in iteration tools. Like enumerate, zip is also its own iterator, so we have to call it again to iterate again:

```
>> Z = zip((1, 2, 3), (10, 20, 30)) >> Z <zip object at 0x101236240> >> I = iter(Z) >> next(I) (1, 10) >> next(I) (2, 20) >> I is Z True >> list(Z) [(3, 30)] >> list(zip((1, 2, 3), (10, 20, 30))) [(1, 10), (2, 20), (3, 30)]
```

中文翻译

正如我们在上一章看到的，遍历字典键的通常方式是 for 循环：

```
>>> D = dict(a=1, b=2) >>> for key in D: # 字典是隐式可迭代的
print(key, D[key]) a 1 b 2
```

这之所以有效，仅仅因为字典是可迭代对象，其迭代器会在 for 这样的迭代工具中自动一次返回一个键：

```
>>> I = iter(D) # 这意味着它们支持该协议
>>> next(I)'a' >>> next(I)'b' >>> next(I) StopIteration
```

迭代协议也是我们在 REPL 中必须用 list 调用包裹 range 结果、才能一次看到所有值的原因。非序列的可迭代对象一次只返回一个结果，而不是返回一个物理列表：

```
>>> R = range(5) >>> R range(0, 5) >>> I = i
```

```

ter(R) # 用迭代协议产生结果>>> next(I) 0 >>> next(I)
1 >>> list(range(5)) # 或者用 list() 一次性收集所有结果[
0, 1, 2, 3, 4] 注意：这里的 list 调用在迭代自动发生的场景
中——比如 for 循环内部——是不需要的，也不应该使用
。但为了一次性显示所有值，它是必需的；当需要类似列表
的行为，或者需要对通常按需产生结果的对象进行多次扫描
时，它也可能是必需的（后面会详述）。现在你对这个协议
有了更好的理解，也应该能明白上一章介绍的 enumerate
工具为什么会那样工作了： >>> E = enumerate('text') #
enumerate 也是一个可迭代对象>>> E <enumerate obj
ect at 0x1010ab880> >>> I = iter(E) >>> next(I) # 用
迭代协议产生结果(0,'t') >>> next(I) # 或者用 list() 强制
生成运行(1,'e') >>> list(enumerate('text')) [(0,'t'), (1,'e'
), (2,'x'), (3,'t')] 与 range 不同，enumerate 内建函数的
结果是它自己的迭代器，很像文件。这意味着每次调用只支
持一次扫描，iter 是可选的（后面还会详述）： >>> R =
range(5) >>> iter(R) is R False >>> E = enumerate('t
ext') >>> iter(E) is E True >>> next(E) (0,'t') 上一章讲
过的 zip 内建函数在迭代工具中的工作方式与此相同。和 e
numerate 一样，zip 也是它自己的迭代器，所以想再次迭
代就必须重新调用它： >>> Z = zip((1, 2, 3), (10, 20, 30
)) >>> Z <zip object at 0x101236240> >>> I = iter(Z
) >>> next(I) (1, 10) >>> next(I) (2, 20) >>> I is Z Tru
e >>> list(Z) [(3, 30)] >>> list(zip((1, 2, 3), (10, 20, 30
))) [(1, 10), (2, 20), (3, 30)]

```

深度理解

·核心概念：四个典型可迭代对象的"协议身份"各不相同：

字典是 iterable (遍历默认给键) ; range 是 iterable (支持多遍、可重复 iter) ; enumerate 和 zip 是"自身即迭代器" (单遍、用完即废) 。

·底层实现: zip 对象在 C 层持有多个指向输入迭代器的引用和一个"结果槽位"。next(Z) 会依次对每个输入迭代器调用 next, 把结果打包成元组; 任何一个输入耗尽 (抛 StopIteration) , zip 就把这个异常原样传递出去, 宣告自己结束——所以 list(Z) 在 Z 已取到 (2, 20) 后只剩 [(3, 30)]。enumerate 对象内部保存一个计数器和一个迭代器, 每次 next 同时递增计数。

·为什么这样设计: range 支持多遍是因为它是"可计算"的 (给定起止和步长, 任意位置的值都能 $O(1)$ 算出) , Python 因此可以安全地提供无限个独立迭代器; zip/enumerate 是"有状态推进器", 状态必须被独占, 所以单遍是诚实的设计——这也迫使程序员养成"需要再扫一遍就重新构造"的习惯。

·生活例子: range 像一座带数字标注的阶梯 (每一级都能随时从头再走) ; zip 像两台并排的传送带+包装机 (产品与包装必须同步拿取, 拿完即停, 想重来只能重新开机) 。

·初学者误区: ① 在 for 循环里多写 list(range(5))——for 内部本来就会迭代, 包一层列表纯属浪费内存; ② 对已经消费过的 zip 结果再 list(Z) 只得到空列表或残留项, 以为出了 bug——其实对象本身耗尽; ③ 忘记 zip/enumerate 的结果可以 .next() 但不能下标、没有长度。

·实际问题：zip 的"短截止"行为（最短输入决定输出长度）和 enumerate 的"自动编号"是 Python 处理"并行遍历"与"带索引遍历"的标准答案，几乎每个真实程序都会用到。

代码分析

```
>>> Z = zip((1, 2, 3), (10, 20, 30)) # zip
>>> next(I) #
(1, 10)
>>> I is Z # zip
True
>>> list(Z) #
[(3, 30)]
>>> list(zip((1, 2, 3), (10, 20, 30))) #
[(1, 10), (2, 20), (3, 30)]
```

解释器如何处理：zip(...) 只创建对象、保存输入元组与迭代器，不产生任何结果——这是惰性 (lazy) 求值。首次 next(I) 才真正向两个输入元组的迭代器要值。list(Z) 会反复 next(Z) 直到 StopIteration，把收集到的结果放进新列表。注意中间 next(I) 已经消耗了前两个元素，所以 list(Z) 只剩 (3, 30)；想重新扫描，唯一办法是再调用一次 zip(...) 构造新对象。

设计思想：惰性 + 单遍是"组合爆炸"的克星——zip 不复制数据，只在输入之间做"同步拉取"，多个迭代工具嵌套（见 14.8.2）时内存占用依然恒定为常数级别。

14.8.2 迭代器嵌套 (Iterator Nesting)

英文原文

More interestingly, `zip` is both iteration tool and iterable: it iterates over its arguments' results, but also returns an iterable object with an iterator for its own results. In the following, for example, it's a tool that receives results from two range iterables, but its own results are produced on demand as well. In other words, there are three iterables and two levels of iteration at work here:

```
>> Z = zip(range(1, 4), range(10, 40)) >> next(Z) (1, 10) >> next(Z) (2, 11) >> next(Z) (3, 12) >> list(zip(range(1, 4), range(10, 40))) [(1, 10), (2, 11), (3, 12)]
```

In fact, iterators can be nested arbitrarily. Because `enumerate` is both iteration tool and object too, in the following, `range`, `enumerate`, and `zip` all produce their results on demand, and `list` makes all the dances run:

```
>> list(enumerate(range(1, 4))) [(0, 1), (1, 2), (2, 3)] > list(zip(enumerate(range(1, 4)), enumerate(range(5, 8)))) [((0, 1), (0, 5)), ((1, 2), (1, 6)), ((2, 3), (2, 7))]
```

Similarly, the following enumerates the zipped results of ranges—but only when requested by `for`:

```
>> for x in enumerate(zip(range(1, 4), range(5, 8))): print(x) (0, (1, 5)) (1, (2, 6)) (2, (3, 7))
```

There are three levels of iterables in this, all deferring their results until they are activated. In practice, this works naturally, but nothing happens in such code until a tool like `list` or `for` asks for results at the top. In response, all the actors here return just one result at a time, to minimize memory requirements and avoid delays.

中文翻译

更有意思的是，`zip` 既是迭代工具又是可迭代对象：它迭代其参数的结果，同时也返回一个带有自己的迭代器、可迭代的对象。比如下面这个例子：它是一个从两个 `range` 可迭代对象接收结果的工具，但它自己的结果也是按需产生的。换句话说，这里有三层可迭代对象、两个级别的迭代在同时工作：

```
>>> Z = zip(range(1, 4), range(10, 40)) >>> next(Z) (1, 10) >>> next(Z) (2, 11) >>> next(Z) (3, 12) >>> list(zip(range(1, 4), range(10, 40))) [(1, 10), (2, 11), (3, 12)]
```

事实上，迭代器可以任意嵌套。因为 `enumerate` 同样既是迭代工具又是对象，在下面的代码中，`range`、`enumerate` 和 `zip` 全都按需产生结果，而 `list` 让所有“舞步”真正跳起来：

```
>>> list(enumerate(range(1, 4))) [(0, 1), (1, 2), (2, 3)] >>> list(zip(enumerate(range(1, 4)), enumerate(range(5, 8)))) [((0, 1), (0, 5)), ((1, 2), (1, 6)), ((2, 3), (2, 7))]
```

类似地，下面这段代码枚举（`enumerate`）`range` 的 `zip` 结果——但只在 `for` 请求时才真正执行：

```
>>> for x in enumerate(zip(range(1, 4), range(5, 8))): print(x) (0, (1, 5)) (1, (2, 6)) (2, (3, 7))
```

这里有三层可迭代对象，全部推迟产生结果，直到被激活。在实践中，这一切自然运

转，但在这类代码中，在 `list` 或 `for` 这样的工具在顶层请求结果之前，什么都不会发生。作为响应，这里的所有“演员”一次只返回一个结果，以最小化内存需求、避免延迟。

深度理解

·核心概念：迭代工具可以“互为输入输出”，形成任意深度的流水线（pipeline）。`zip/enumerate` 既是工具（消费别人的迭代）又是原料（提供自己的结果给别人消费）。

·底层实现：每次 `next` 请求从上到下层层触发：`list` 向 `zip` 要一个元组 → `zip` 向两个 `enumerate` 各要一个元素 → `enumerate` 再向各自的 `range` 迭代器要整数并配计数。一层返回后，结果逐级组装、返回顶层。整条链在任意时刻内存里只有“一个结果在途”。

·为什么这样设计：惰性（lazy）求值是函数式思想的核心：计算被推迟到需要的最后一刻。这让“无限序列”（如 `itertools.count()`）也能参与有限组合——只要消费端在有限的请求后就停止。

·生活例子：像餐厅流水线：顾客点菜（`list` 请求）→ 厨师问配菜（`zip` 要两种原料）→ 备菜员递上（`range/enumerate` 供应）。没有订单（没有 `next`）时，整个后厨零工作；订单来了，一道菜一道菜地出。

·初学者误区：① 以为 `list(zip(...))` 里的 `zip` 会“先把 `range` 变成列表”——不会，`range` 元素是逐个被拉的；② 多层嵌套的结果类型难读（`[((0, 1), (0, 5)), ...]` 是“元组的元组”），读时从最内层括号拆解；③ 误以为组合会“一次性算完”——恰恰相反，是“一次算一个”。

·实际问题：这正是 `itertools` 模块所有工具（`chain`、`product`、`groupby` 等）能无缝组合的机制基础，也是“大数据不落内存”编程范式的核心技法。

14.8.3 函数式可迭代对象：map 与 filter (Functional Iterables: map and filter)

英文原文

Like `range`, `enumerate`, and `zip`, the `map` and `filter` built-ins produce their results individually to conserve space and avoid pauses. Like `enumerate` and `zip`, these tools also are both iterable tools and iterable objects themselves: they scan other iterables, and produce their own results on demand.

Unlike other iterables we've met, though, `map` and `filter` apply function calls instead of expressions, so their complete story requires function-coding skills we won't gain until the next part of the book. Still, we can preview their fundamentals here using built-in functions without having to code new functions of our own.

For example, the `map` built-in, which made a brief cameo appearance in Chapter 8 (and has nothing directly to do with mappings like dictionaries!), calls a provided function for each item in a provided iterable, and returns the collected results as another iterable. In the following, it applies the `ord` built-in to collect cha

racter code points:

```
>> ord('p')
```

```
# Return a single character's code point 112 >> M =  
map(ord,'py3')
```

```
# map returns an iterable, not a list >> M
```

```
# Runs ord(x) for every x in iterable <map object at 0  
x101227550> >> next(M)
```

```
# Iterating manually: exhausts results 112 >> next(M)  
121 >> next(M) 51 >> next(M) StopIteration
```

As usual, you can force results with list if you must treat them as a list, and for automates iterations:

```
>> list(map(ord,'py3'))
```

```
# Force a real list - only if needed [112, 121, 51] >>  
M = map(ord,'py3')
```

```
# Must call again to scan again >> for x in M: print(x,  
end='')
```

```
# Iteration tools auto call next() 112 121 51
```

The filter built-in, which we met momentarily in Chapter 12 and will study more fully in the next part of this book, is analogous. It returns items in an iterable for which a passed-in function returns True. In the following, we're leveraging concepts we've already learned—True includes nonempty and nonzero objects, the bool built-in returns a single object's truth value, and the str string's isdigit method is true for all-digit

text:

```
>> filter(bool, ['lp6e','', 2024]) <filter object at 0x101227850> >> list(filter(bool, ['lp6e','', 2024]))
```

```
# Collect "true" items ['lp6e', 2024] >> list(filter(str.isdigit, ['lp6e','2024']))
```

```
# Collect all-digit strings ['2024']
```

Like most of the tools discussed in this section, `filter` both accepts an iterable to process and returns an iterable to generate results. It doesn't do any work until code like a `for` loop asks it to. As a preview, both `map` and `filter` can be emulated roughly with list comprehensions and more closely with Chapter 20's generators or expressions; but to grok the following code in full, we have to await this chapter's presentation of comprehensions coming up soon:

```
>> [ord(x) for x in 'py3'] [112, 121, 51] >> [x for x in ['lp6e','2024'] if x.isdigit()] ['2024']
```

中文翻译

和 `range`、`enumerate`、`zip` 一样，`map` 和 `filter` 内建函数逐个产生结果，以节省空间、避免停顿。和 `enumerate`、`zip` 一样，这两个工具自身也既是迭代工具又是可迭代对象：它们扫描其他可迭代对象，并按需产生自己的结果。不过，与我们见过的其他可迭代对象不同，`map` 和 `filter` 应用的是函数调用而不是表达式，所以它们完整的故事需要函

数编码技能——这要等到本书下一部分才能获得。尽管如此，我们可以在这里用内建函数预览它们的基本原理，不必自己编写新函数。例如，map 内建函数（在第 8 章短暂客串过，而且与字典那样的映射（mapping）没有任何直接关系！）对提供的可迭代对象中的每一项调用一个提供的函数，并把收集到的结果作为另一个可迭代对象返回。下面它应用 ord 内建函数来收集字符码点：

```
>>> ord('p') # 返回
单个字符的码点112 >>> M = map(ord,'py3') # map 返回
可迭代对象，而非列表>>> M # 对可迭代对象中的每个
x 运行 ord(x) <map object at 0x101227550> >>> next
(M) # 手动迭代：会耗尽结果112 >>> next(M) 121 >>
> next(M) 51 >>> next(M) StopIteration 照例，如果你
必须把结果当作列表处理，可以用 list 强制取出所有结果；
for 则自动完成迭代： >>> list(map(ord,'py3')) # 强制生
成真实列表——仅在需要时[112, 121, 51] >>> M = map
(ord,'py3') # 想再次扫描必须重新调用>>> for x in M: pr
int(x, end='') # 迭代工具自动调用 next() 112 121 51 filt
er 内建函数与它类似——我们第 12 章曾短暂见过它，下一
部分还会更全面地学习。它返回可迭代对象中让传入函数
返回 True 的那些项。下面的例子用到了我们已经学过的概念——True 包含非空、非零对象；bool 内建函数返回单
个对象的真值；str 字符串的 isdigit 方法对全数字文本为
真： >>> filter(bool, ['lp6e','', 2024]) <filter object at
0x101227850> >>> list(filter(bool, ['lp6e','', 2024])) #
收集"为真"的项['lp6e', 2024] >>> list(filter(str.isdigit, ['
lp6e','2024'])) # 收集全数字字符串['2024'] 与本节讨论的
大多数工具一样，filter 既接受一个待处理的可迭代对象，
又返回一个用来产生结果的可迭代对象。在 for 循环这样的
```

代码请求它之前，它什么也不做。预告一下：map 和 filter 都可以用列表推导式粗略模拟，用第 20 章的生成器表达式则能更精确地模拟；但要完全理解下面的代码，我们得等本章稍后对推导式的讲解：

```
>>> [ord(x) for x in 'py3'] [112, 121, 51]
>>> [x for x in ['lp6e', '2024'] if x.isdigit()] ['2024']
```

深度理解

·核心概念：map(func, iterable) = "对每个元素套用函数"；filter(func, iterable) = "留下函数为真的元素"。它们把"遍历 + 变换/筛选"这两个高频操作封装成惰性对象，是函数式编程（functional programming）风格在 Python 内建工具中的代表。

·底层实现：map 对象保存函数指针和输入迭代器；next 时先向输入迭代器取一个值，再调用 C 层函数调用机制执行 func 并返回结果。filter 对象则循环调用 func 直到返回真值为止（可能跳过多个元素），耗尽输入后抛 StopIteration。两者都不预计算、不缓存。

·为什么这样设计：用"函数作为参数"（一等公民函数）让变换逻辑可复用、可组合——同一个 map(ord, ...) 可以作用于任何可迭代对象。注意 Lutz 特意提醒：map 与字典的"映射"概念无关，只是名字撞车。

·生活例子：map 像流水线上的喷漆机：每个零件（元素）过一遍机器（函数），出来都是喷好漆的（变换结果）；filter 像安检门：人（元素）逐个过，符合条件（函数返回 True）的放行，其余拦下。

·初学者误区：① 直接打印 `map(ord,'py3')` 只能看到 `<map object ...>`——必须 `list()` 或 `for` 才能看到结果，这是“惰性”的直接体现；② 以为 `map` 能当字典用——它只是“映射函数到元素”；③ 想再次遍历结果必须重新构造对象（单遍）；④ 与列表推导式相比，`map` 要求“函数”，表达式更受限——这是它的局限。

·实际问题：`filter(bool, data)` 是 Python 里一行去掉所有假值（空串、0、None）的惯用法；`map(str.strip, lines)` 批量清洗数据。第 19、20 章将把它发扬光大。

14.8.4 多遍扫描与单遍扫描的可迭代对象（Multiple-pass versus Single-pass Iterables）

英文原文

We've noted a few times that some iterables that don't allow multiple scans are their own iterables. Since this is a subtle difference that can impact the way you'll use them, it's worth a separate callout here.

In particular, the range built-in's result, along with objects like dictionaries and lists, differs from other built-ins described in this section. They are not their own iterators (you must make one with `iter` when iterating manually), and they support multiple iterators (each remembers its position independently):

```
>> R = range(3)
```

```
# range allows multiple iterators >> next(R) TypeError: range object is not an iterator >> I1 = iter(R) >> next(I1) 0 >> next(I1) 1 >> I2 = iter(R)
```

```
# Two iterators on one range >> next(I2) 0 >> next(I1)
```

```
# I1 is at a different spot than I2 2
```

By contrast, the results of `enumerate`, `zip`, `map`, `filter`, as well as `open` for files, are their own iterators, because none of these tools support multiple active iterations on the same call result. Because of this, the `iter` call is optional for stepping through such objects' results (though harmless: their `iter` is themselves), and we must call these tools again to begin a fresh iteration from the start:

```
>> Z = zip((1, 2, 3), (10, 11, 12)) >> I1 = iter(Z) >> I2 = iter(Z)
```

```
# Two iterators on one zip >> next(I1) (1, 10) >> next(I1) (2, 11) >> next(I2)
```

```
# But I2 is at same spot as I1! (3, 12) >> M = map(ord, 'py3')
```

```
# Ditto for map (and others) >> I1, I2 = iter(M), iter(M) >> next(I1), next(I1) (112, 121) >> next(I2) 51 >> L = [0, 1, 2]
```

```
# But lists (and others) do many scans >> I1, I2 = iter(L), iter(L) >> next(I1), next(I1) (0, 1) >> next(I2)
```

Multiple active scans, like range 0

When we code our own iterable objects with classes later in the book (Chapter 30), you'll see that multiple iterators are usually supported by returning new objects for the `iter` call; a single iterator generally means an object returns itself and supports `next` directly.

In Chapter 20, you'll also find that generator functions and expressions behave like `map` and `zip` instead of `range` in this regard, supporting just a single active iteration scan. Also in that chapter, you'll see some subtle implications of single-pass iterators in loops that attempt to scan multiple times—code that treats these as lists may fail without manual list conversions.

中文翻译

我们已多次提到，一些不允许多次扫描的可迭代对象本身就是自己的迭代器。由于这是一个微妙的差异，会直接影响你的使用方式，值得单独拿出来讲一讲。特别是，`range` 内建函数的结果，以及字典、列表这样的对象，与本节描述的其他内建工具不同。它们不是自己的迭代器（手动迭代时必须用 `iter` 创建一个），并且支持多个迭代器（每个迭代器独立记住自己的位置）：

```
>>> R = range(3) # range 允许多个迭代器
>>> next(R)
TypeError: range object is not an iterator
>>> I1 = iter(R)
>>> next(I1)
0
>>> next(I1)
1
>>> I2 = iter(R) # 同一个 range 上的两个迭代器
>>> next(I2)
0
>>> next(I1)
2
```

I1 与 I2 处于不同的位置

相比之下，`enumerate`、`zip`、`map`、`filter` 的结果，以及 `open` 产生的文件对象，是它们自己的迭代器，因为这些工

具都不支持对同一个调用结果进行多次同时进行的迭代。正因如此，`iter` 调用对遍历这类对象的结果是可选的（虽然无害：它们的 `iter` 就是它们自己），而要从头开始一次全新的迭代，我们必须重新调用这些工具：

```
>>> Z = zip((1, 2, 3), (10, 11, 12))
>>> I1 = iter(Z)
>>> I2 = iter(Z) # 同一个 zip 上的两个迭代器
>>> next(I1) (1, 10)
>>> next(I1) (2, 11)
>>> next(I2) # 但 I2 与 I1 处于同一位置! (3, 12)
>>> M = map(ord, 'py3') # map (以及其他工具)
同理>>> I1, I2 = iter(M), iter(M)
>>> next(I1), next(I1) (112, 121)
>>> next(I2) 51
>>> L = [0, 1, 2] # 但列表 (及其他对象) 支持多次扫描
>>> I1, I2 = iter(L), iter(L)
>>> next(I1), next(I1) (0, 1)
>>> next(I2) # 多个同时进行的扫描, 和 range 一样
```

0 在本节后面的第 30 章，当我们用类编写自己的可迭代对象时，你会看到：支持多个迭代器通常意味着 `iter` 调用返回新对象；而单个迭代器通常意味着对象返回自身并直接支持 `next`。在第 20 章你还会发现，生成器函数和表达式在这方面表现得像 `map` 和 `zip` 而不是 `range`——只支持单次活跃的迭代扫描。同样在那章，你会看到单遍迭代器在试图多次扫描的循环中的一些微妙后果——把这类对象当列表用的代码，如果不手动转成列表，可能会失败。

深度理解

·核心概念：可迭代对象分成两大阵营——多遍型（`range`、`list`、`dict`：`iter(x) is x` 为 `False`，可同时开多个独立迭代器）与单遍型（文件、`zip`、`enumerate`、`map`、`filter`：`iter(x) is x` 为 `True`，同一对象只有一个“游标”）。

·底层实现：多遍型对象的 `tpiter` 每次返回新建的迭代器 (`listiterator` / `rangeiterator` / `dictkeyiterator`)，各自带独立位置字段；单遍型对象的 `tpiter` 返回 `self`，位置状态就存在对象本体的字段里，天然共享。这就是 `I2 = iter(Z)` 后 `next(I2)` 直接跳过前两个元素的根本原因。

·为什么这样设计：多遍型的数据是“随机可访问”的（按下标或按公式可取任意位置），多开几个游标零成本；单遍型的数据（文件流、打包拉取的 `zip`）本质是“消耗式”的——取了就没了，无法凭空恢复现场，诚实暴露“只能一次”才符合直觉。

·生活例子：多遍型像书（多个书签可以同时在这书的不同页）；单遍型像传呼机语音信箱（听完一条就删一条，想重听只能等对方再留言）。把 `zip` 当列表用，就像听完留言还想“再听一遍”——已经没有数据了。

·初学者误区：① 对 `zip/map` 的结果写两个 `for` 循环，第二个循环发现什么都没有——单遍耗尽了；② 用 `next(R)` 直接对 `range` 调用报 `TypeError`——`range` 是 `iterable` 不是 `iterator`；③ 在一个 `zip` 上创建两个“迭代器”以为能并行遍历——它们共享同一个游标，不是独立的；④ 把单遍对象传给 `sorted/list` 之外还需要二次扫描的代码而不做 `list()` 转换。

·实际问题：调试时“我先看看内容再处理”的思路在单遍对象上行不通——正确姿势是“一次消费、立即决策”，或先 `list()` 固化再多次使用。

14.8.5 Python 标准库中的可迭代对象 (Standard-library Iterables in Python)

英文原文

Finally, while out of scope here, and technically part of its standard library instead of its language, Python provides additional tools that support the iteration protocol and thus may also be used in for loops and other iteration tools. For instance, shelves (an access-by-key filesystem for Python objects), as well as the results of `os.popen` (a tool for reading the output of shell commands), are iterables that can be processed with the full set of iteration tools. The standard directory (a.k.a. folder) walker in Python, `os.walk`, is iterable too, but we'll save details and an example until Chapter 20's coverage of this tool's basis—generators and `yield`. Ultimately, all such tools implement the `iter/next` interface defined by the iteration protocol. We don't normally see this machinery because for and its kin run it for us automatically to step through results. In fact, everything that scans left to right in Python employs the iteration protocol in the same way—including the topic of the next section.

中文翻译

最后，虽然超出了本书这里的范围，而且严格来说属于标准库而非语言本身，Python 还提供了其他支持迭代协议的工

具，因此它们也可以用在 for 循环和其他迭代工具中。例如，shelves（一种按键访问 Python 对象的文件系统）、以及 os.popen 的结果（一种读取 shell 命令输出的工具），都是可迭代对象，可以用全套迭代工具处理。Python 标准的目录（也就是文件夹）遍历工具 os.walk 也是可迭代的，但细节和示例要留到第 20 章讲解这个工具的基础——生成器（generators）和 yield——时再说。归根结底，所有这些工具都实现了迭代协议定义的 iter/next 接口。我们通常看不到这套机制，因为 for 和它的同类会替我们自动运行它来逐步取结果。事实上，Python 中一切从左到右扫描的东西都以同样的方式使用迭代协议——包括下一节的主题。

深度理解

- 核心概念：迭代协议的“适用面”远超核心语言——标准库里凡是“从头到尾取数据”的工具（shelf 遍历键、popen 逐行读输出、os.walk 递归目录），全都实现同一套协议。学会协议，等于学会了标准库一半工具的用法。
- 底层实现：shelve.Shelf 实现了 keys() 并支持迭代；os.popen 返回的文件流对象沿袭文件迭代器；os.walk 是生成器函数（yield 驱动的惰性对象，每次 next 才向下递归目录）——它同时也是第 20 章“生成器”主题的引子。
- 为什么这样设计：协议把“数据从哪里来”彻底抽象掉：目录树的递归遍历、shell 输出、键值库的键集合，在 for 眼里统统只是“一个能吐结果的东西”。这种统一性是 Python“一致的语言模型”的体现（呼应第 1 章 fit your brain）。

- 生活例子：迭代协议像"快递驿站的自助取件码"——不管包裹来自哪个仓库（文件、数据库、目录树），你（迭代工具）都凭同一个取件码流程取件。
 - 初学者误区：把"语言特性"和"标准库工具"看成两套东西——实际上协议打通了二者；另注意 `os.walk` 是生成器，一次遍历是"消耗式"的，反复遍历要重新调用。
 - 实际问题：实际项目中"逐行读命令输出"、"遍历所有子目录文件"、"按 key 遍历持久化数据"这些高频任务，都只是"拿着 `for` 套一个标准库迭代器"而已。
-

14.9 推导式 (Comprehensions)

英文原文

Now that we've seen how the iteration protocol works, let's turn to one of its most common use cases. Together with `for` loops, list comprehensions are one of the most prominent contexts in which the iteration protocol is applied. In the previous chapter, we learned how to use `range` to change a list as we step across it:

```
>>> L = [1, 2, 3, 4, 5]
>>> for i in range(len(L)): L[i] += 10
>>> L
```

`[11, 12, 13, 14, 15]` This works, but as mentioned there, it may not be the optimal "best practice" approach in Python. Today, the list comprehension expression makes many such prior coding patterns obsolete. Here, for example, we can replace the loop with a single expression that produces the desired

result list: `>>> L = [x + 10 for x in L]` `>>> L` `[21, 22, 23, 24, 25]` The net result is similar, but it requires less coding on our part and is likely to run substantially faster—in fact, it's often twice as fast as tested in CPython 3.12. The list comprehension isn't exactly the same as the for loop statement version because it makes a new list object, which might matter if there are multiple references to the original list, but it's close enough for most applications and is a common and convenient enough approach to merit a closer look here.

中文翻译

现在我们看到了迭代协议是如何工作的，接下来看它最常见的用例之一。与 for 循环一起，列表推导式 (list comprehensions) 是迭代协议被应用的最突出场景之一。上一章我们学会了如何用 range 在遍历列表的同时修改它：`>>> L = [1, 2, 3, 4, 5]` `>>> for i in range(len(L)): L[i] += 10` `>>> L` `[11, 12, 13, 14, 15]` 这个方法有效，但正如上一章提到的，它可能不是 Python 中最优的“最佳实践”做法。如今，列表推导式表达式让许多此类旧式编码模式过时了。例如在这里，我们可以用一个产生期望结果列表的单一表达式替换整个循环：`>>> L = [x + 10 for x in L]` `>>> L` `[21, 22, 23, 24, 25]` 最终结果类似，但它需要的编码更少，而且很可能运行得明显更快——事实上，在 CPython 3.12 上测试，它常常快一倍 (twice as fast)。列表推导式与 for 循环语句版本并不完全相同，因为它创建的是一个新的列表对象——如果原列表有多个引用，这一点可能很重要——但对大多数应用来说，两者足够接近，而且推导式是一种常见

又方便的写法，值得在这里仔细看看。

深度理解

·核心概念：推导式是"生成新结果集合"的表达式式写法。它把"建空列表 → 循环 → 逐个 append"三步压成一行，且不修改原列表（新建对象）。

·底层实现：推导式的迭代在解释器内部以专门的字节码序列运行（LISTAPPEND 指令、专属的快速局部变量槽），不需要经过 Python 级的属性查找和 list.append 方法调用，所以通常比等价 for 循环快约 2 倍。它内部的求值作用域也与外部隔离（Python 3 中推导式有独立的局部作用域）。

·为什么这样设计："对每个元素做变换并收集结果"是编程中最常见的任务之一（map 的教义），Python 选择用内建语法而非函数来表达，是为了可读性和速度；新建列表则避免了循环内修改正在迭代的容器这一危险操作。

·生活例子：for 循环改列表像"给旧家具逐个重新刷漆"（原地翻新）；推导式像"照旧家具尺寸做一批新家具"（新建一套，旧的不动）。新建副本意味着多个引用互不干扰。

·初学者误区：① 以为推导式是在原列表上操作——它是新建列表，`L = [x+10 for x in L]` 是把新列表重新绑定到 L；② 误以为推导式一定更快——有 if 过滤、极短列表等场景差异很大（见章末 Speed disclaimer）；③ 把"循环改值"和"生成新列表"两种需求混为一谈。

·实际问题：数据清洗、特征工程（对每个样本做变换）、批量格式化——凡是"输入一个集合、输出一个同规模变换集合"的活，推导式都是首选。

14.10 列表推导式基础 (List Comprehension Basics)

英文原文

The list comprehension was introduced in Chapter 4, and it's been demoed often. Syntactically, its syntax is derived from a construct in set theory notation that applies an operation to each item in a set, but you don't have to know set theory to use this tool. In Python, most people find that a list comprehension simply looks like a backward for loop. To get a handle on the syntax, let's dissect the prior section's example in more detail: `L = [x + 10 for x in L]` List comprehensions are written in square brackets because they are ultimately a way to construct a new list. They begin with an arbitrary expression that we make up, which uses a loop variable that we make up (`x + 10`). That is followed by what you should now recognize as the header of a for loop, which names the loop variable, and an iterable object (`for x in L`). To run the expression, Python executes an iteration across `L` inside the interpreter, assigning `x` to each item in turn, and collects the results of running the items through the expression `o`

n the left side. The result list we get back is exactly what the list comprehension says—a new list containing $x + 10$, for every x in L . Technically speaking, list comprehensions are never really required because we can always build up a list of expression results manually with for loops that append results as we go:

```
>>> res = []
>>> for x in L: res.append(x + 10)
>>> res
```

 [31, 32, 33, 34, 35] In fact, this is exactly what the list comprehension does internally (using internal equivalents, of course). However, list comprehensions are more concise to write, and widely useful in Python programs because building result lists is such a common task. Moreover, depending on your Python and code, list comprehensions might run much faster than manual for loop statements (often 2X as stated earlier) because their iterations are performed at the speed of optimized (and usually compiled) code inside the interpreter, rather than with manual Python code. Especially for larger data sets, there is often a major performance advantage to using this expression.

中文翻译

列表推导式在第 4 章就介绍过了，此后也经常演示。从语法上讲，它的语法源自集合论表示法 (set theory notation) 中的一种构造——对集合中每个元素应用一个操作——但使用这个工具并不需要你懂集合论。在 Python 中，大多数人觉得列表推导式看起来就像一个倒过来的 for 循环 (backward for loop)。为了把握语法，我们更细致地解剖

上一节的例子：`L = [x + 10 for x in L]` 列表推导式写在方括号里，因为它归根结底是构造新列表的方式。它以一个我们自拟的任意表达式开头，该表达式使用一个我们自拟的循环变量（`x + 10`）。紧接着是你现在应该能认出来的 for 循环头：它命名循环变量和一个可迭代对象（`for x in L`）。为了运行这个表达式，Python 在解释器内部执行对 L 的一次迭代，依次把 x 赋给每一项，并收集这些项经过左侧表达式运行后的结果。我们拿回的结果列表正是列表推导式所描述的——一个新列表，包含对 L 中每个 x 的 `x + 10`。严格来说，列表推导式从来不是必需之物，因为我们总可以用 for 循环手动构建表达式结果列表，边循环边 append：

```
>>> res = [] >>> for x in L: res.append(x + 10) >>> res
```

[31, 32, 33, 34, 35] 事实上，这正是列表推导式在内部做的事情（当然，用的是内部等价物）。然而，列表推导式写起来更简洁，在 Python 程序中应用广泛，因为构建结果列表是如此常见的任务。此外，取决于你的 Python 版本和代码，列表推导式可能比手动 for 循环语句快得多（正如前面所说，通常是 2 倍），因为它们的迭代以解释器内部优化的（通常已编译的）代码的速度执行，而不是手动 Python 代码。特别是对更大的数据集，使用这个表达式往往有巨大的性能优势。

深度理解

·核心概念：推导式语法 = [表达式 for 变量 in 可迭代对象]。方括号表示“产出列表”，表达式用循环变量拼出每个结果，for 子句与普通 for 循环头同构。它是“循环 + 收集”的压缩记法。

·底层实现：CPython 将推导式编译为专门的字节码：先 BUILDLIST（或直接复用预分配列表），循环体只含"求值表达式 → LISTAPPEND"两步；循环变量使用独立的快速局部变量槽（compiler 为推导式分配专属的 slot），避免污染外部命名空间，也省去全局/闭包查找。相比 `res.append(x+10)` 每轮要做的"属性查找 + 方法调用"（两条额外字节码），推导式天然更快。

·为什么这样设计：Lutz 点出推导式源自集合论描述法 $\{f(x) \mid x \in S\}$ ——数学里"对集合整体施加操作"的记法被 Python 语法化。选方括号是因为"结果是列表"。Python 3 把推导式改成独立作用域，是修复 Python 2 中"循环变量泄漏到外部"的设计缺陷。

·生活例子：推导式像"批量化生产订单"：`[x+10 for x in L]` 是一张清单"每件货都加价 10 元"；`for+append` 是"逐个拿货、逐个改价、逐个放上新货架"。结果一样，清单式写法更快更省事。

·初学者误区：① 把循环变量想成"预定义"的——它是推导式内部自动赋值的；② 以为推导式能代替所有 `for` 循环——它只做"收集结果"这一件事，纯副作用（打印、写文件）的循环不该用推导式；③ 忘记推导式的作用域隔离——`[x for x in L]` 之后，外层变量 `x` 不受影响（Python 3）。

·实际问题：性能差距在十万级以上的数据上极为显著（约 2 倍）；同时推导式读起来"先结果后来源"，与自然语言"我要这些元素的这些变换"一致，可读性更好。

代码分析

```
L = [1, 2, 3, 4, 5]
L = [x + 10 for x in L]    # L  x  x+10
print(L)                  # [11, 12, 13, 14, 15]

res = []                  #
for x in L:
    res.append(x + 10)    # append
```

解释器如何处理：[x + 10 for x in L] 被编译器翻译成约 6 条字节码的循环：GETITER（对 L 调用内部等价于 iter() 的操作）→ FORITER（每次取一个元素，StopIteration 时跳到循环末尾）→ STOREFAST（把元素存入推导式专属的局部槽 x）→ BINARYOP +（执行 x + 10）→ LIST APPEND（把结果追加进正在构造的列表，该指令直接操作列表对象的底层数组，不经过 Python 方法调用）。而手动版每轮还要执行 LOADMETHOD res.append（属性查找）、CALL（方法调用）等 4~5 条额外字节码。这就是 2 倍差距的来源。注意推导式执行后，外部作用域里没有新增或改变名为 x 的变量。

设计思想：推导式把"迭代机制"交给解释器的专用指令（快），把"变换规则"交给程序员（自由），语法上"先写结果、再写来源"，贴合"我要什么"的思维顺序。

14.11 列表推导式与文件（List Comprehensions and Files）

英文原文

Let's work through another common application of list comprehensions to explore them in more detail. Recall that the file object has a `readlines` method that loads the file into a list of line strings all at once:

```
>>> f = open('data.txt') >>> lines = f.readlines() >>> lines
['Testing file IO\n', 'Learning Python, 6E\n', 'Python 3.12\n']
```

This works as we saw earlier, but the lines in the result all include the newline character (`\n`) at the end. For many programs, the newline character gets in the way—we have to be careful to avoid double-spacing when printing, and so on. It would be nice if we could get rid of these newlines all at once, wouldn't it? Any time we start thinking about performing an operation on each item in a sequence, we're in the realm of list comprehensions. For example, assuming the variable `lines` is as it was in the prior interaction, the following code does the job by running each line in the list through the string `rstrip` method to remove whitespace on the right side (a `line[:-1]` slice would work, too, but only if we can be sure all lines are properly `\n` terminated, and this may not always be the case for the last line in a file):

```
>>> lines = [line.rstrip() for line in lines] >>> lines
['Testing file IO', 'Learning Python, 6E', 'Python 3.12']
```

This works as planned. Because list comprehensions are an iteration tool just like for loop statements, though, we don't even have to open the file ahead of time. If we open it inside the expression, the list comprehension will automatically use the iteration protocol we met earlier in this chapter. That is, it will read one line from the file at a time by calling the file's next handler method, run the line through the `rstrip` expression, and add it to the result list. Again, we get what we ask for—the `rstrip` result of a line, for every line in the file:

```
>>> lines = [line.rstrip() for line in open('data.txt')]
>>> lines ['Testing file IO', 'Learning Python, 6E', 'Python 3.12']
```

This expression does a lot implicitly, but we're getting a lot of work for free here—Python scans the file line by line and builds a list of operation results automatically. It's also an efficient way to code this operation: because most of this work is done inside the Python interpreter, it's likely faster than an equivalent for statement. Just as importantly, its use of file iterators means that it won't load the file into memory all at once, like `readlines` does. Again, especially for large files, the advantages of list comprehensions can be significant. Besides their efficiency, list comprehensions are also remarkably expressive. In our example, we can run any string operation on a file's lines as we iterate. To illustrate, here's the list comprehension equivalent

ent to the file iterator uppercase example we met earlier, along with a few other representative operations to sample the possibilities:

```
>>> [line.upper() for line in open('data.txt')] ['TESTING FILE IO\n','LEARNING PYTHON, 6E\n','PYTHON 3.12\n']
```

```
>>> [line.rstrip().upper() for line in open('data.txt')] ['TESTING FILE IO','LEARNING PYTHON, 6E','PYTHON 3.12']
```

```
>>> [line.split() for line in open('data.txt')] [['Testing', 'file', 'IO'], ['Learning', 'Python,', '6E'], ['Python', '3.12']]
```

```
>>> [line.replace("\n", '!') for line in open('data.txt')] ['Testing file IO!','Learning Python, 6E!','Python 3.12!']
```

```
>>> [('Py' in line, line.split()[0]) for line in open('data.txt')] [(False, 'Testing'), (True, 'Learning'), (True, 'Python')]
```

Recall that the method chaining in the second of these examples works because string methods return a new string, to which we can apply another string method. The last of these shows how we can also collect multiple results, as long as they're wrapped in a collection like a tuple or list. One fine point here: recall from Chapter 9 that file objects close themselves automatically in CPython when garbage-collected if still open. Hence, these list comprehensions will also automatically close the file when their temporary file object

is garbage-collected after the expression runs. Outside CPython, though, you may want to code these to close manually if this is run in a loop, to ensure that file resources are freed immediately: open before the comprehension, and close after. See Chapter 9 for more on file close calls if you need a refresher on this.

中文翻译

让我们再走一遍列表推导式的一个常见应用，以更详细地探索它。回忆一下：文件对象有一个 `readlines` 方法，它一次性把文件加载成一个行字符串列表：

```
>>> f = open('data.txt')
>>> lines = f.readlines()
>>> lines
['Testing file I
O\n','Learning Python, 6E\n','Python 3.12\n']
```

正如我们之前看到的，这个方法有效，但结果中的每一行都带末尾的换行符（`\n`）。对很多程序来说，这个换行符很碍事——打印时我们必须小心避免隔行打印，等等。如果能一次性去掉这些换行符就好了，不是吗？每当我们开始思考“对序列中的每个元素做一个操作”时，我们就进入了列表推导式的领域。例如，假设变量 `lines` 还和上次交互时一样，下面的代码用 `rstrip` 方法处理列表中的每一行以去除右侧空白，就完成了这个任务（用 `line[:-1]` 切片也可以，但前提是我们能确定所有行都正确地以 `\n` 结尾——而文件的最后一行未必总是如此）：

```
>>> lines = [line.rstrip() for line in lines]
>>> lines
['Testing file IO','Learning Python, 6E','Python 3.12']
```

一切按计划运行。但由于列表推导式和 `for` 循环语句一样是迭代工具，我们甚至不必提前打开文件。如果我们在表达式内部打开它，列表推导式会自动使用本章前面介绍的迭代协议。也就是说，它会通过调用文件的 `next`

处理方法一次从文件读取一行，把该行送入 `rstrip` 表达式，再把它加入结果列表。我们又一次得到我们要求的——文件中每一行的 `rstrip` 结果：

```
>>> lines = [line.rstrip() for line in open('data.txt')] >>> lines ['Testing file IO','Learning Python, 6E','Python 3.12']
```

这个表达式隐式地做了很多事，但我们在这里免费得到了大量工作——Python 自动逐行扫描文件并构建操作结果列表。这也是编码此操作的高效方式：因为大部分工作都在 Python 解释器内部完成，它很可能比等价的 `for` 语句更快。同样重要的是，它使用文件迭代器，意味着它不会像 `readlines` 那样一次性把整个文件加载进内存。再说一次，特别是对大文件，列表推导式的优势可能是显著的。除了高效，列表推导式还格外富有表现力。在我们的例子中，可以在迭代时对文件的每一行运行任意字符串操作。为了说明这一点，下面是前面遇到的"文件迭代器大写"示例的列表推导式版本，连同几个有代表性的操作，一起体验一下各种可能性：

```
>>> [line.upper() for line in open('data.txt')] ['TESTING FILE IO\n','LEARNING PYTHON, 6E\n','PYTHON 3.12\n'] >>> [line.rstrip().upper() for line in open('data.txt')] ['TESTING FILE IO','LEARNING PYTHON, 6E','PYTHON 3.12'] >>> [line.split() for line in open('data.txt')] [['Testing','file','IO'], ['Learning','Python','6E'], ['Python','3.12']] >>> [line.replace('\n','!') for line in open('data.txt')] ['Testing file IO!','Learning Python, 6E!','Python 3.12!'] >>> [( 'Py' in line, line.split()[0]) for line in open('data.txt')] [(False,'Testing'), (True,'Learning'), (True,'Python')] 回想一下，第二个例子中的方法链式调用（chaining）之所以可行，是因为字符串方法返回一个新的字符串，我们可以对返回值
```

继续应用另一个字符串方法。最后一个例子展示了如何同时收集多个结果——只要把它们包进元组或列表这样的集合中。这里有一个细节：回忆第 9 章，文件对象如果仍然打开，会在被垃圾回收（garbage-collected）时由 CPython 自动关闭自己。因此，这些列表推导式在表达式运行后，当临时文件对象被垃圾回收时，也会自动关闭文件。不过，在 CPython 之外的实现中，如果这在循环中运行，你可能要手动关闭：在推导式之前打开，之后关闭，以确保文件资源被立即释放。如需复习文件关闭调用的知识，参见第 9 章。

深度理解

- 核心概念：推导式是完整意义上的迭代工具——`open('data.txt')` 直接写进推导式，文件迭代器自动逐行供数。这打通了“文件处理”与“集合变换”两个世界：不用先读出，边读边变换边收集。
- 底层实现：`[line.rstrip() for line in open('data.txt')]` 的字节码序列与 14.10 相同，只是每次 FORITER 触发的是文件对象的 `next`（C 层读一行）。文件对象在推导式结束后成为临时对象，引用计数归零即被回收；CPython 的 `TextIOWrapper` 在析构时会自动调用 `close()` 释放操作系统文件句柄。
- 为什么这样设计：把 `open` 放进表达式，实现了“从源到结果的单行流水线”，比“先 `f=open(...)` 再 `for` 再 `f.close()`”简洁得多；惰性逐行读取又规避了 `readlines` 的内存爆炸。这正是迭代协议“省内存”哲学在推导式里的兑现。

·生活例子：readlines 像把整个书架搬回家再逐本挑书；推导式像在书店现场一本本翻、挑中意的直接装袋（读一行、处理一行、放进行李箱），箱子（内存）永远只有一书之重。

·初学者误区：① 在推导式里对文件做 readlines() 再用——多此一举且浪费内存；② 用 line[:-1] 去换行符——文件最后一行可能没有 \n，会误删最后一个字符，rstrip() 才稳妥；③ 以为表达式里的文件“开着没人关”——CPython 会自动回收关闭，其他实现（如 PyPy、Jython）则要手动管理；④ 一次推导式试图同时遍历多个文件多个循环——嵌套没问题，但单行表达式可读性会快速崩塌。

·实际问题：日志清洗、CSV 首轮扫描、配置文件解析——“逐行变换 + 条件收集”是所有数据管道的第一公里，推导式（后续还有生成器表达式）就是这段路的铺路机。

代码分析

```
>>> lines = [line.rstrip() for line in open('data.txt')] # ...
>>> lines
['Testing file IO', 'Learning Python, 6E', 'Python 3.12']

>>> [line.rstrip().upper() for line in open('data.txt')] # ...
['TESTING FILE IO', 'LEARNING PYTHON, 6E', 'PYTHON 3.12']

>>> [line.split() for line in open('data.txt')] # ...
[['Testing', 'file', 'IO'], ['Learning', 'Python,', '6E'], ['Pytho...

>>> [('Py' in line, line.split()[0]) for line in open('data.txt')]...
[(False, 'Testing'), (True, 'Learning'), (True, 'Python')]
```

解释器如何处理：第一行：open('data.txt') 创建文件对

象（此时文件尚未读取），FORITER 通过文件迭代器每次取一行（内部缓冲区 + next），line 存入推导式局部槽，调用 `str.rstrip()`（C 实现，遍历字符串尾部空白生成新字符串），LISTAPPEND 入列表。三行数据读完，第四次 next 抛 `StopIteration`，循环结束，临时文件对象引用计数归零、析构关闭文件。方法链 `line.rstrip().upper()` 是两次 C 层调用：先得到去尾的新串，再对新串执行 `upper`，产生第二个新串。最后一个例子中表达式是元组（'Py' in line, line.split()[0]）——推导式只关心"表达式求值的结果"，元素可以是任何类型，所以用元组即可"打包"多个结果。

设计思想：推导式是"表达式即产出"——产出物可以是标量、元组、甚至嵌套列表，表达力来自左侧表达式可以任意组合方法调用；而内存安全来自右侧迭代源的惰性。两者结合，就是"高效 + 表现力"的配方。

14.12 列表推导式扩展语法 (Extended List Comprehension Syntax)

英文原文

Handy as they may already seem, list comprehensions can be even richer in practice, and even constitute a sort of iteration mini-language in their fullest forms. Let's take a quick look at their extra syntax tools here

中文翻译

尽管列表推导式看起来已经很方便了，在实践中它们还可以更丰富，在它们最完整的形式下，甚至构成了一种迭代迷你语言（iteration mini-language）。这里我们快速看看它们额外的语法工具。

深度理解

- 核心概念：推导式的完整语法 = 任意多个 for 子句 + 每个 for 后可带一个 if 子句。[表达式 for ... for ... if ...] 构成"扁平化的嵌套循环 + 过滤"。
- 底层实现：多个 for 子句编译成嵌套的循环字节码（外层 for 是主导循环，内层 for 每轮外层都要重新初始化）；if 子句编译成 POPJUMPIFFALSE 跳转指令，条件为假就跳过表达式直接进入下一轮迭代。
- 为什么这样设计：这种语法的来源同样是集合论记法 $\{f(x) \mid x \in A, P(x)\}$ （带条件约束的描述法）。Python 把"循环 + 分支"揉进表达式，是因为这两个结构在"批量产生数据"时总是成对出现。
- 生活例子：[x+y for x in 'abc' for y in '123'] 像"排列组合配对"：外层循环是选字母，内层循环是配数字，每个字母配完所有数字才轮到下一个字母——所以结果是 a1 a2 a3 b1 b2 b3 c1 c2 c3。
- 初学者误区：① 把多个 for 的嵌套顺序搞反——最前面的 for 是外层循环，循环次数最少的那层；② 推导式写得太长（超过一行或两层以上嵌套）时可读性骤降——Lutz

明确建议"超出此复杂度，就用简单的 for 语句"；③ 以为 if 只能放在最后——每个 for 后面都能跟一个 if，作用域覆盖到它后面的所有子句。

·实际问题：笛卡尔积、矩阵展平、带条件的数据筛选——这些需求一行推导式就能表达，但"看懂"与"写对"之间的沟壑正是本节要填的。

14.12.1 过滤子句：if (Filter Clauses: if)

英文原文

As one particularly useful extension, the for loop nested in a comprehension expression can have an associated if clause to filter out of the result items for which the test is not true. (It's really a filter in, but it works out the same.) For example, suppose we want to repeat the prior section's file-scanning example, but we need to collect only lines that begin with the letters L or P (perhaps the first character on each line is an action code of some sort). Adding a simple if filter clause to our expression does the trick:

```
>>> lines = [line.rstrip() for line in open('data.txt') if
line[0] in 'LP'] >>> lines
['Learning Python, 6E', 'Python 3.12']
```

Here, the if clause checks each line read from the file to see whether its first character matches; if not, the line is omitted from the result list, and the iteration continues. This is a fairly big expression, but it's easy to

o understand if we translate it to its simple for loop statement equivalent:

```
>>> res = [] >>> for line in open('data.txt'): if line[0] in 'LP': res.append(line.rstrip())  
>>> res ['Learning Python, 6E', 'Python 3.12']
```

In general, we can always translate a list comprehension to a for statement by appending as we go and further indenting each successive part. The converse isn't true: for statements are more general and can address additional roles out of scope for comprehensions, but the latter's results collection is a very common task. This for statement equivalent works, but it takes up four lines instead of one and may run slower. In fact, you can squeeze a substantial amount of logic into a list comprehension when you need to—the following works like the prior but selects only lines that end in a digit (before the newline at the end), by filtering with a more sophisticated expression on the right side (which uses `[-1:]` instead of `[-1]` to handle files with blank lines empty after `rstrip`):

```
>>> [line.rstrip() for line in open('data.txt') if line.rstrip()[-1:].isdigit()] ['Python 3.12']
```

As another if filter example, the first result in the following gives the total lines in a text file, and the second strips whitespace on both ends to omit blank lines in the tally in just one line of code:

```
>>> fname = 'data-blank-lines.txt'>>> len(open(fname).readlines()) # All lines 5
>>> len([line for line in open(fname) if line.strip() != '']) # Nonblank lines 3
```

中文翻译

一个特别有用的扩展是：推导式表达式中嵌套的 for 循环可以带一个关联的 if 子句，把测试不为真的项从结果中过滤掉（实际上这是“过滤进来”（filter in），但效果相同）。例如，假设我们要重做上一节扫描文件的例子，但只需要收集以字母 L 或 P 开头的行（也许每行第一个字符是某种动作代码）。给表达式加上一个简单的 if 过滤子句就搞定了：

```
>>> lines = [line.rstrip() for line in open('data.txt') if line[0] in 'LP']
```

>>> lines ['Learning Python, 6E', 'Python 3.12']

这里，if 子句检查从文件读出的每一行，看它的第一个字符是否匹配；如果不匹配，该行就从结果列表中被省略，迭代继续。这是一个相当大的表达式，但如果把它翻译成简单的 for 循环语句等价形式，就很容易理解了：

```
>>> res = []
>>> for line in open('data.txt'): if line[0] in 'LP': res.append(line.rstrip())
```

>>> res ['Learning Python, 6E', 'Python 3.12']

一般来说，我们总可以把列表推导式翻译成 for 语句：边循环边 append，并把后续的部分逐级缩进。反过来就不成立：for 语句更通用，可以承担推导式范围之外的更多角色，但推导式的结果收集（results collection）是一个非常常见的任务。这个 for 语句等价形式有效，但它占了四行而不是一行，而且可能更慢。事实上，需要时你可以在列表推导式里塞进相当多的逻辑——下

面这个和前面效果类似，但只选择以数字结尾的行（在行尾换行符之前），通过在右侧用更复杂的表达式过滤实现（用 `[-1:]` 而不是 `[-1]`，以处理 `rstrip` 后为空的空行）：

```
>>> [line.rstrip() for line in open('data.txt') if line.rstrip()[-1:].isdigit()]
```

['Python 3.12'] 作为另一个 `if` 过滤的例子，下面第一个结果给出文本文件的总行数，第二个结果用一行代码去除两端空白、排除空白行后再计数：

```
>>> fname = 'data-blank-lines.txt'
>>> len(open(fname).readlines())
# 所有行5
>>> len([line for line in open(fname) if line.strip() != ''])
# 非空行3
```

深度理解

·核心概念：`if` 子句是“选择器”：只有条件为真的元素才进入结果。它等价于 `for` 循环体里的 `if ...: res.append(...)`，是 `filter` 函数的语法级化身。

·底层实现：`if` 条件被编译成一条 `POPJUMPIFFALSE` 指令：每轮迭代先求值条件表达式，为假则跳过结果表达式直接进入下一轮 `FORITER`。注意 `line.rstrip()[-1:].isdigit()` 中 `rstrip()` 被调用了两次（一次在条件里，一次在结果表达式里）——代码重复执行，功能正确但略浪费，更优写法是把 `rstrip` 提前（如再用一层变量，或用 14.13 的生成器表达式，或干脆接受这个成本）。

·为什么这样设计：“先筛选、再变换”是数据处理的标准次序，`if` 子句让“筛选”不必写成独立的 `filter` 调用，保持单表达式的完整性；而“翻译成 `for` 语句”的等价性证明（`append` + 缩进）让任何推导式都可读、可调试。

·生活例子：if line[0] in 'LP' 像流水线上的自动质检：每个产品（行）先过质检（首字符检查），不合格的直接滑走，合格的才继续加工（rstrip）装箱（进入结果列表）。

·初学者误区：① 在 if 条件里做和结果表达式里相同的昂贵操作（如重复 rstrip）；② 用 [-1] 判断"以数字结尾"——空行 rstrip() 后是"，"[-1] 直接 IndexError，[-1:] 返回" 则安全地判为 False，这是书中特意演示的边界陷阱；③ 把 if 放在错误的 for 之后，导致过滤范围不是自己想要的。

·实际问题：日志过滤、配置项筛选、按行统计（总行数 v s 非空行）——"一行代码 + 条件"是数据探查（data profiling）的标准姿势。

代码分析

```
>>> lines = [line.rstrip() for line in open('data.txt') if line...
>>> lines
['Learning Python, 6E', 'Python 3.12']

>>> fname = 'data-blank-lines.txt'
>>> len(open(fname).readlines())                                     #
5
>>> len([line for line in open(fname) if line.strip() != '']) # ...
3
```

解释器如何处理：第一条：每轮迭代，先求值 if 后的条件 line[0] in 'LP'——line[0] 取首字符（一个单字符字符串），in 'LP' 是字符串成员测试（C 层字符扫描），条件为假时 POPJUMPIFFALSE 直接跳到下一轮迭代，line.rstrip() 根本不会执行；为真才执行 rstrip 并 LISTAPPEND。第

二条：open(fname).readlines() 一次性读出全部 5 行放入列表再 len；而 len([...推导式]) 只把通过 line.strip() != "" 的行收集起来再数——推导式本身仍是一次一行地迭代文件（strip() 会创建新字符串，即使只是为了比较）。两个表达式展示了"全量加载 vs 惰性过滤"的差别：前者内存为整个文件，后者峰值内存只有"已收集的行"。

设计思想：推导式把 map（变换）与 filter（筛选）两个操作合二为一，且顺序清晰：先 for（取源）→ if（选）→ 表达式（变换）。这是数据管道的"一行版"。

14.12.2 嵌套循环：for（Nested Loops: for）

英文原文

List comprehensions can become even more complex if we need them to—for instance, they may contain nested loops, coded as a series of for clauses. In fact, their full syntax allows for any number of for clauses, each of which can have an optional associated if clause. For example, the following builds a list of the concatenation of $x + y$ for every x in one string and every y in another. It effectively collects all the ordered combinations of the characters in two strings: >>> [x + y for x in 'abc' for y in '123'] ['a1','a2','a3','b1','b2','b3','c1','c2','c3'] Again, one way to understand this expression is to convert it to statement form by indenting its parts. The following is an equivalent, but likely slower, alternative way to achieve the same effect (it's t

he same as the first nested for example in the prior chapter, but builds a list of results instead of simply printing them): >>> res = [] >>> for x in 'abc': for y in '123': res.append(x + y) >>> res ['a1','a2','a3','b1','b2','b3','c1','c2','c3'] Beyond this complexity level, though, list comprehension expressions can sometimes become too compact for their own good. In general, they are intended for simple types of iterations; for more involved work, a simpler for statement structure will probably be easier to understand and modify in the future. As usual in programming, if something is difficult for you to understand, it's probably not the best idea.

中文翻译

如果我们需要，列表推导式可以变得更复杂——例如，它们可以包含嵌套循环，写成一系列 for 子句。事实上，它们的完整语法允许任意数量的 for 子句，每个子句都可以带一个可选的关联 if 子句。例如，下面的代码构建了一个 x + y 拼接结果的列表，其中 x 遍历一个字符串中的每个字符，y 遍历另一个字符串中的每个字符。它有效地收集了两个字符串中所有字符的有序组合（ordered combinations）：>>> [x + y for x in 'abc' for y in '123'] ['a1','a2','a3','b1','b2','b3','c1','c2','c3'] 再次强调，理解这个表达式的方法之一是把它按缩进转换成语句形式。下面这个等价但可能更慢的替代方案能达到同样效果（它和上一章第一个嵌套 for 的例子相同，只是把结果构建成列表而不是直接打印）：>>> res = [] >>> for x in 'abc': for y in '123': res.append(

```
x + y) >>> res ['a1','a2','a3','b1','b2','b3','c1','c2','c3']
```

不过，超过这个复杂度级别，列表推导式表达式有时会紧凑得过了头。总的来说，它们是为简单类型的迭代设计的；对更复杂的工作，一个简单的 for 语句结构可能更容易理解、也更方便将来修改。编程中一贯如此：如果某样东西让你难以理解，它很可能不是好主意。

深度理解

·核心概念：多个 for 子句 = 嵌套循环的扁平记法。执行顺序与缩进版的 for 完全一致：最外层 for 每取一个元素，内层 for 就完整跑一遍。

·底层实现：编译器生成两层循环字节码；外层 FORITER 每轮将 x 存入一个局部槽，内层 FORITER 将 y 存入另一个局部槽，最内层执行 x + y 并 LISTAPPEND。内层迭代器在每轮外层循环开始时被重新获取（GETITER 重新出现），这正是"a 配完 1、2、3 再轮到 b"的实现机制。

·为什么这样设计：笛卡尔积式（有序组合）的需求无处不在（坐标生成、测试用例矩阵、字符串配对），数学记号 $\{x+y \mid x \in A, y \in B\}$ 的直接转写让推导式成为最贴近"数学直觉"的 Python 结构。

·生活例子：像中式餐桌的"转盘"：外层是坐主位的你（x），每次轮到你，服务员（内层循环）就给你完整上一圈菜（y 的全部）；下一轮换人，菜再上一圈。

·初学者误区：① 写反嵌套顺序——`[x+y for y in '123' for x in 'abc']` 结果顺序完全不同（先 1a 1b 1c...）；② 嵌套三层以上或表达式超长时硬用推导式——Lutz 的忠告：

"难以理解 = 不是好主意", 此时该换回 for 语句; ③ 忽略复杂度——两个千元素列表的嵌套推导式会产生 100 万个元素, 内存与时间双双爆炸。

·实际问题: 生成网格坐标、枚举所有参数组合 (配合 `iter tools.product` 的语法级替代)、二维矩阵展平——在这些场景, 嵌套推导式是标准答案, 但规模失控前要学会收手。

代码分析

```
>>> [x + y for x in 'abc' for y in '123']    # x y
['a1', 'a2', 'a3', 'b1', 'b2', 'b3', 'c1', 'c2', 'c3']

#
>>> res = []
>>> for x in 'abc':
        # x  a, b, c
        for y in '123':
            # y  1, 2, 3
            res.append(x + y)
```

解释器如何处理: 推导式的两层 FORITER 严格保持"外层每轮一元素, 内层跑完整轮"的顺序: $x='a'$ 时内层产出 'a1'、'a2'、'a3', $x='b'$ 时内层再次从头获取 '123' 的迭代器并产出 'b1'、'b2'、'b3'.....内层迭代器每轮外层都被重新获取, 这正是嵌套语义的关键。字节码层面, 内层循环体是 " $x+y$ 求值 + LISTAPPEND", 外层只负责推进 x 与重新初始化内层迭代器。手动版本每轮多出 `res.append` 的属性查找与方法调用开销, 所以稍慢。

设计思想: 推导式的可读性来源于"执行顺序 = 书写顺序"——从左到右读 `for x in 'abc' for y in '123'`, 就是"先 x 后 y 、先外后内"的嵌套循环。这个与缩进版完全一致的语

义约定，使推导式翻译回 for 语句成为机械而可靠的调试手段。

14.13 推导式悬念 (Comprehensions Cliff-Hanger)

英文原文

Because comprehensions are generally better taken in multiple doses, we'll cut this story short here for now. We'll revisit list comprehensions in Chapter 20 in the context of functional programming tools, and will define their syntax more formally and explore additional examples there. As you'll find, comprehensions turn out to be just as related to functions as they are to looping statements. List comprehensions are also related to—and predate—the set and dictionary comprehensions introduced in this book's prior part, as well as the generator expression you'll meet later that produces items on request instead of building a list. All share the same syntax, but are coded slightly differently and produce different sorts of stuff:

```
[x + 10 for x in L if x > 0] # List comprehension {x + 10 for x in L if x > 0} # Set comprehension {x: x + 10 for x in L if x > 0} # Dictionary comprehension (x + 10 for x in L if x > 0) # Generator expression
```

We'll put the last three of these to work on files briefly in the next section. To better expand this plotline to generators, though, we have to move on to this book's next part. > NOTE: Speed disclaimer (速度声明) : As a blanket qualification for all performance claims in this book, the relative speed of code depends much on the exact code tested and version of Python used, and is prone to vary and change. For example, list comprehensions have been consistently twice as fast as corresponding for loops on most tests for all CPython versions through 3.12, but may be just marginally quicker on some tests, and perhaps even slower when if filter clauses are used. You'll learn how to time your own code and Python in Chapter 21. For now, keep in mind that absolutes in performance benchmarks are as elusive as consensus in open source projects.

中文翻译

因为推导式通常适合“分几次服用”，我们先把这个故事在这里收个尾。我们会在第 20 章函数式编程工具的语境下重新讨论列表推导式，在那里正式定义它们的语法并探索更多示例。你会发现，推导式与函数的关系，和与循环语句的关系一样密切。列表推导式还与本书前一部分介绍的集合（set）和字典（dictionary）推导式，以及你稍后会遇到的生成器表达式（generator expression）相关——生成器表达式按请求产生元素，而不是构建列表——并且比它们更早出现。它们共享同一套语法，但编码方式略有不同，产出不同种类的东西：`[x + 10 for x in L if x > 0]` # 列表推导式{

`x + 10 for x in L if x > 0` # 集合推导式
`{x: x + 10 for x in L if x > 0}` # 字典推导式
`(x + 10 for x in L if x > 0)` # 生成器表达式

我们会在下一节把后三种短暂地应用到文件上。不过，要把这条故事线更充分地扩展到生成器，我们得进入本书的下一部分。> 注意：速度声明：作为本书所有性能论断的总括性限定条件：代码的相对速度很大程度上取决于被测试的确切代码和所用 Python 版本，而且容易变动。例如，对所有直到 3.12 的 CPython，列表推导式在大多数测试中一直比对应的 for 循环快两倍，但在某些测试中可能只是略微更快，在使用 if 过滤子句时甚至可能更慢。你将在第 21 章学会如何给自己的代码和 Python 计时。眼下请记住：性能基准测试中的“绝对结论”，就像开源项目中的“共识”一样难得。

深度理解

- 核心概念：同一套推导式语法有四个变体——方括号（列表）、花括号无冒号（集合）、花括号带冒号（字典）、圆括号（生成器表达式）。括号决定产出类型。
- 底层实现：四种形式编译为不同的字节码序列：列表用 `LISTAPPEND`，集合用 `SETADD`，字典用 `MAPADD`，生成器表达式则根本不立即执行——它生成一个生成器对象，字节码在每次 `next` 时才运行。生成器表达式因此是四种中唯一“惰性到极致”的：不占结果内存。
- 为什么这样设计：语法统一降低了学习成本（学一个等于会四个）；而 if 过滤、嵌套 for 等扩展语法在四种形式中全部通用。Lutz 说“推导式与函数一样关系密切”——因为第 20 章会把它们当作“函数式工具”（`map/filter` 的语法

替代品) 重新诠释。

·生活例子：四种括号像四种包装盒——同一批产品（同样的迭代与变换逻辑），装进纸箱（列表）、网兜（集合）、带标签的抽屉（字典）、或“按需取货的预约单”（生成器表达式）。

·初学者误区：① 圆括号推导式 $(x+10 \text{ for } x \text{ in } L)$ 不是“元组推导式”——Python 没有元组推导式，它是生成器表达式；② 以为四种形式性能相同——生成器表达式按需产元素，列表推导式一次性占内存；③ 忽略速度声明的限定：带 if 过滤时推导式可能并不比 for 快，“两倍速”不是铁律。

·实际问题：大数据集上“只要前几个结果”时用生成器表达式；结果需要去重用集合推导式；构建索引映射用字典推导式——工具选型取决于产出形态。

14.14 迭代工具 (Iteration Tools)

英文原文

Later in this book, you'll learn how user-defined classes can implement the iteration protocol too. Because of this, it's sometimes important to know which built-in tools make use of it—any tool that employs the iteration protocol will automatically work on any built-in type or user-defined class that provides it. This section closes out this chapter with a summary and sort of "grand finale" of tools in this domain.

So far, we've mostly seen iterators at work in the context of the for loop statement, because this part of the book is focused on statements. Keep in mind, though, that every built-in tool that scans from left to right across collection objects uses the iteration protocol. This includes the for loops we've seen:

```
>> for line in open('data.txt'):
print(line.upper(), end='') TESTING FILE IO LEARNING
PYTHON, 6E PYTHON 3.12
```

But also much more. For instance, the prior section's list comprehensions and the map built-in function we met earlier use the same protocol as their for loop cousin. When applied to a file, they both leverage the file object's iterator automatically to scan line by line, fetching an iterator with iter and calling next each time through:

```
>> uppers = [line.upper() for line in open('data.txt')]
>> uppers ['TESTING FILE IO\n','LEARNING PYTHON,
6E\n','PYTHON 3.12\n'] >> list(map(str.upper, open('
data.txt'))) ['TESTING FILE IO\n','LEARNING PYTHON,
6E\n','PYTHON 3.12\n']
```

As we saw earlier, the map built-in applies a function call to each item in an iterable object. map is similar to a list comprehension but is more limited because it requires a function instead of an arbitrary expression. It also returns an iterable object, so we must wrap it in a list call to force it to give us all its values at once.

Because map, like the list comprehension, is related to both for loops and functions, watch for its revival in Chapter 20.

Many of Python's other built-ins process iterables, too. We've seen how zip combines items from iterables, enumerate pairs items in an iterable with relative positions, and filter selects items for which a function is true. In addition, sorted sorts items in an iterable, and reduce (now oddly relegated to a module) runs pairs of items in an iterable through a function.

All of these accept iterables, and zip, enumerate, and filter also return an iterable like map. Here they are in action running the file's iterator automatically to read line by line:

```
>> sorted(open('data.txt')) ['Learning Python, 6E\n', 'Python 3.12\n', 'Testing file IO\n'] >> list(zip(range(99), open('data.txt'))) [(0, 'Testing file IO\n'), (1, 'Learning Python, 6E\n'), (2, 'Python 3.12\n')] >> list(enumerate(open('data.txt'))) [(0, 'Testing file IO\n'), (1, 'Learning Python, 6E\n'), (2, 'Python 3.12\n')] >> list(filter(bool, open('data.txt'))) ['Testing file IO\n', 'Learning Python, 6E\n', 'Python 3.12\n'] >> import functools, operator >> functools.reduce(operator.add, open('data.txt')) 'Testing file IO\nLearning Python, 6E\nPython 3.12\n'
```

All of these are iteration tools, but they have unique roles. We met zip and enumerate in this chapter; filter and reduce are in Chapter 19's functional program

ming domain, so we'll defer their details for now; the point to notice here is their use of the iteration protocol for files and other iterables.

We first saw the `sorted` function used here at work in Chapter 4, and we used it in Chapter 8. `sorted` is a built-in that employs the iteration protocol—it's like the original `list` sort method, but it returns the new sorted list as a result and runs on any iterable object. Notice that, unlike `map` and others, `sorted` returns an actual list instead of an iterable.

Per the prior chapter's closer, its reversed cohort returns an iterable but does not run the iteration protocol.

In general, though, everything in Python's built-in toolset that scans object is defined to use the iteration protocol on their subject. This even includes tools such as the `list` and `tuple` built-in functions (which build new objects from iterables), and Chapter 7's `string` `join` method (which makes a new string by putting a substring between strings in an iterable). Hence, these will also work on an open file and automatically read one line at a time:

```
>> list(open('data.txt')) ['Testing file IO\n','Learning Python, 6E\n','Python 3.12\n'] >> tuple(open('data.txt')) ('Testing file IO\n','Learning Python, 6E\n','Python 3.12\n') >> '&&'.join(open('data.txt')) 'Testing file IO\n&&Learning Python, 6E\n&&Python 3.12\n'
```

Even some tools you might not expect fall into this category. For example, Chapter 11's sequence assignment (original and starred), the `in` membership test, slice assignment, the list's `extend` method, and single-star literal unpacking also leverage the iteration protocol to scan, and thus read a file by lines automatically:

```
>> a, b, c = open('data.txt')

# Sequence assignment >> >> b, c ('Learning Python
, 6E\n','Python 3.12\n') >> a, b = open('data.txt')

# Extended-unpacking assignment >> a, b ('Testing f
ile IO\n', ['Learning Python, 6E\n','Python 3.12\n']) >
> 'Python 2.7\n' in open('data.txt')

# Membership test False >> 'Python 3.12\n' in open('
data.txt') True >> L = [11, 22, 33, 44]

# Slice assignment >> L[1:3] = open('data.txt') >> L [
11,'Testing file IO\n','Learning Python, 6E\n','Python
3.12\n', 44] >> L = [11] >> L.extend(open('data.txt'))

# list.extend method >> L [11,'Testing file IO\n','Lear
ning Python, 6E\n','Python 3.12\n'] >> L = [11, open('
data.txt'), 44]

# List-literal unpacking >> L [11,'Testing file IO\n','Le
arning Python, 6E\n','Python 3.12\n', 44]
```

Remember that `extend` iterates automatically, but `append` does not—though you can use the latter to add an iterable to a list without iterating, and iterate across it later:

```
>> L = [11] >> L.append(open('data.txt')) >> list(L[-1]) ['Testing file IO\n','Learning Python, 6E\n','Python 3.12\n']
```

Nor does the iteration grand finale end here. In the prior chapter, we saw that the built-in dict call accepts an iterable zip result too. For that matter, so does the set call, as well as the set and dictionary comprehension expressions we met earlier and will revisit later:

```
>> set(open('data.txt')) {'Python 3.12\n','Learning Python, 6E\n','Testing file IO\n'} >> {line for line in open('data.txt')} {'Python 3.12\n','Learning Python, 6E\n','Testing file IO\n'} >> {ix: line for ix, line in enumerate(open('data.txt'))} {0:'Testing file IO\n', 1:'Learning Python, 6E\n', 2:'Python 3.12\n'}
```

As noted, both set and dictionary comprehensions support the extended syntax of list comprehensions we met earlier in this chapter, including if tests:

```
>> {line for line in open('data.txt') if line[0] in 'LP'} {'Python 3.12\n','Learning Python, 6E\n'} >> {ix: line for (ix, line) in enumerate(open('data.txt')) if line[0] in 'LP'} {1:'Learning Python, 6E\n', 2:'Python 3.12\n'}
```

Like the list comprehension, both of these scan the file line by line and pick out lines that begin with the specific letters. They also happen to build sets and dictionaries in the end, but we get a lot of work "for free" by combining file iteration and comprehension syntax.

ax. In the next part of this book, you'll meet a relative of comprehensions—generator expressions—that deploys the same syntax and works on iterables too, but it is also iterable itself:

```
>> list(line.upper() for line in open('data.txt')) ['TESTING FILE IO\n','LEARNING PYTHON, 6E\n','PYTHON 3.12\n']
```

Other built-in functions support the iteration protocol as well, but frankly, some are harder to cast in interesting examples related to files. For example, the `sum` call computes the sum of all the numbers in any iterable; the `any` and `all` built-ins return `True` if any or all items in an iterable are `True`, respectively; and `max` and `min` return the largest and smallest item in an iterable, respectively.

Like `reduce`, all of the tools in the following examples accept any iterable as an argument and use the iteration protocol to scan it, but return a single result:

```
>> sum(range(5))  
  
# Punt (requires numbers) 10  
>> any(open('data.txt'))  
)  
  
# Any/all lines true (nonempty) True  
>> all(open('data.txt'))  
  
# Mostly pointless for files True  
>> max(open('data.txt'))
```

```
# Line with highest string value'Testing file IO\n'>>  
min(open('data.txt'))
```

```
# Use cases wanted!'Learning Python, 6E\n'
```

There's one last iteration tool worth mentioning, although it's a preview of this book's next part: in Chapter 18, you'll learn that a special form can be used in function calls to unpack a collection of values into individual arguments, much as it does in list (and other) literals. As you can probably predict by now, this accepts any iterable too:

```
>> def f(a, b, c):
```

```
# See Part IV print(a, b, c, sep='&') >> f(open('data.txt'))
```

```
# Iterates by lines too! Testing file IO &Learning Python, 6E &Python 3.12
```

In fact, because this argument-unpacking syntax in calls accepts iterables, it's also possible to use the zip built-in to unzip zipped tuples, by making prior or nested zip results arguments for another zip call (warning: you probably shouldn't read the following example if you plan to operate heavy machinery anytime soon!):

```
>> X, Y = (1, 2), (3, 4) >> list(zip(X, Y))
```

```
# Zip tuples: returns an iterable [(1, 3), (2, 4)] >> A, B  
= zip(zip(X, Y))
```

```
# Unzip a zip, really! >> A, B ((1, 2), (3, 4))
```

And that concludes our iteration-tools finale. It's probably not complete, but you probably get the point.

中文翻译

在本书后面，你会学到用户自定义类也可以实现迭代协议。正因如此，有时搞清楚哪些内建工具使用了该协议很重要——任何使用迭代协议的工具，都会自动作用于任何提供了该协议的内建类型或用户自定义类。本节以本领域工具的总结和某种意义上的“压轴大戏”（grand finale）来结束本章。

到目前为止，我们主要在 for 循环语句的语境中看到迭代器在工作，因为本书这部分聚焦于语句。但要记住：每一个从左到右扫描集合对象的内建工具都使用迭代协议。这包括我们见过的 for 循环：

```
>>> for line in open('data.txt'): print(line.upper(), end='') TESTING FILE IO LEARNING PYTHON, 6E PYTHON 3.12
```

但远远不止。例如，上一节的列表推导式和我们之前见过的 map 内建函数，与它们的表亲 for 循环使用同一个协议。应用于文件时，它们都自动利用文件对象的迭代器逐行扫描，用 iter 获取迭代器，每一轮调用 next：

```
>>> uppers = [line.upper() for line in open('data.txt')] >>> uppers ['TESTING FILE IO\n', 'LEARNING PYTHON, 6E\n', 'PYTHON 3.12\n'] >>> list(map(str.upper, open('data.txt'))) ['TESTING FILE IO\n', 'LEARNING PYTHON, 6E\n', 'PYTHON 3.12\n']
```

正如我们之前看到的，map 内建函数对可迭代对象中的每个元素应用一次函数调用。map 与列表推导式类似，但更受限，因为它要求函数而不是任意表达式。它还返回一个可迭代对象，所以我们必须用 list 调用包裹它，才能一次取出所有值。因为

map 和列表推导式一样，既与 for 循环有关又与函数有关，注意它在第 20 章的复兴。Python 的许多其他内建函数也处理可迭代对象。我们已经见过 zip 如何合并多个可迭代对象的元素、enumerate 如何把可迭代对象中的元素与相对位置配对、filter 如何选择函数为真的元素。此外，sorted 对可迭代对象中的元素排序，而 reduce（如今奇怪地被贬到一个模块里）让可迭代对象中的元素两两经过一个函数。所有这些都接受可迭代对象，而 zip、enumerate 和 filter 也像 map 一样返回可迭代对象。下面它们都在运行，自动使用文件的迭代器逐行读取：

```
>>> sorted(open('data.txt')) ['Learning Python, 6E\n','Python 3.12\n','Testing file IO\n'] >>> list(zip(range(99), open('data.txt'))) [(0, 'Testing file IO\n'), (1, 'Learning Python, 6E\n'), (2, 'Python 3.12\n')] >>> list(enumerate(open('data.txt'))) [(0, 'Testing file IO\n'), (1, 'Learning Python, 6E\n'), (2, 'Python 3.12\n')] >>> list(filter(bool, open('data.txt'))) ['Testing file IO\n','Learning Python, 6E\n','Python 3.12\n'] >>> import functools, operator >>> functools.reduce(operator.add, open('data.txt')) 'Testing file IO\nLearning Python, 6E\nPython 3.12\n'
```

所有这些都是迭代工具，但各有独特角色。zip 和 enumerate 我们在本章见过；filter 和 reduce 属于第 19 章函数式编程的领域，细节暂且搁置；这里要注意的是它们对文件和其他可迭代对象使用了迭代协议。我们第一次见到 sorted 函数是在第 4 章，第 8 章又用过它。sorted 是一个使用迭代协议的内建函数——它类似于列表原来的 sort 方法，但返回一个新的排序列表作为结果，并且作用于任何可迭代对象。注意，与 map 等不同，sorted 返回的是真正的 list 而不是可迭代对

象。按照上一章的结尾所说，它的伙伴 `reversed` 返回可迭代对象，但走的不是迭代协议。不过总的来说，Python 内建工具集中一切扫描对象的工具都被定义为对扫描对象使用迭代协议。这甚至包括 `list` 和 `tuple` 内建函数（它们从可迭代对象构建新对象）、以及第 7 章的字符串 `join` 方法（它通过在可迭代对象中的字符串之间插入一个子串来生成新字符串）。因此，这些工具也能作用于一个打开的文件，自动逐行读取：

```
>>> list(open('data.txt')) ['Testing file IO\n', 'Learning Python, 6E\n', 'Python 3.12\n'] >>> tuple(open('data.txt')) ('Testing file IO\n', 'Learning Python, 6E\n', 'Python 3.12\n') >>> '&&'.join(open('data.txt')) 'Testing file IO\n&&Learning Python, 6E\n&&Python 3.12\n'甚至一些你意想不到的工具也属于这一类。例如，第 11 章的序列赋值（原始版和带星号版）、in 成员测试、切片赋值、列表的 extend 方法，以及单星号字面量解包，也都借助迭代协议进行扫描，从而自动按行读取文件：
```

```
>>> a, b, c = open('data.txt') # 序列赋值>>> b, c ('Learning Python, 6E\n', 'Python 3.12\n') >>> a, b = open('data.txt') # 扩展解包赋值>>> a, b ('Testing file IO\n', ['Learning Python, 6E\n', 'Python 3.12\n']) >>> 'Python 2.7\n' in open('data.txt') # 成员测试False >>> 'Python 3.12\n' in open('data.txt') True >>> L = [11, 22, 33, 44] # 切片赋值>>> L[1:3] = open('data.txt') >>> L [11, 'Testing file IO\n', 'Learning Python, 6E\n', 'Python 3.12\n', 44] >>> L = [11] >>> L.extend(open('data.txt')) # list.extend 方法>>> L [11, 'Testing file IO\n', 'Learning Python, 6E\n', 'Python 3.12\n'] >>> L = [11, open('data.txt'), 44] # 列表字面量解包>>> L [11, 'Testing f
```

ile IO\n','Learning Python, 6E\n','Python 3.12\n', 44]

记住：extend 会自动迭代，而 append 不会——不过你可以用后者把可迭代对象不加迭代地添加进列表，以后再迭代它：

```
>>> L = [11] >>> L.append(open('data.txt')) >>> list(L[-1]) ['Testing file IO\n','Learning Python, 6E\n','Python 3.12\n']
```

迭代压轴大戏还不止于此。上一章我们看到，内建 dict 调用也接受可迭代的 zip 结果。同样地，set 调用也接受，我们之前遇到、稍后会再见的集合和字典推导式表达式也是如此：

```
>>> set(open('data.txt')) {'Python 3.12\n','Learning Python, 6E\n','Testing file IO\n'} >>> {line for line in open('data.txt')} {'Python 3.12\n','Learning Python, 6E\n','Testing file IO\n'} >>> {ix: line for ix, line in enumerate(open('data.txt'))} {0:'Testing file IO\n', 1:'Learning Python, 6E\n', 2:'Python 3.12\n'}
```

如前所述，集合和字典推导式都支持本章前面讲过的列表推导式扩展语法，包括 if 测试：

```
>>> {line for line in open('data.txt') if line[0] in 'LP'} {'Python 3.12\n','Learning Python, 6E\n'} >>> {ix: line for (ix, line) in enumerate(open('data.txt')) if line[0] in 'LP'} {1:'Learning Python, 6E\n', 2:'Python 3.12\n'}
```

和列表推导式一样，这两者都逐行扫描文件，挑出以特定字母开头的行。它们碰巧最终构建出集合和字典，但我们通过组合文件迭代与推导式语法，白白获得了大量工作。在本书下一部分，你会见到推导式的近亲——生成器表达式（generator expressions）——它使用同样的语法，同样作用于可迭代对象，但它本身也是可迭代的：

```
>>> list(line.upper() for line in open('data.txt')) ['TESTING FILE IO\n','LEARNING PYTHON, 6E\n','PYTHON 3.12\n']
```

其他内建函数也支持迭代协议，但坦率

地说，有些难以编排成与文件相关的有趣例子。例如，sum 调用计算任何可迭代对象中所有数字之和；any 和 all 内建函数分别返回可迭代对象中“任一”或“全部”项为 True 的布尔值；max 和 min 分别返回可迭代对象中的最大项和最小项。与 reduce 一样，下面例子中的所有工具都接受任何可迭代对象作为参数，并用迭代协议扫描它，但只返回一个单一结果：

```
>>> sum(range(5)) # 搁置（需要数字）10
>>> any(open('data.txt')) # any/all 行是否为真（非空）True
>>> all(open('data.txt')) # 对文件来说基本没意义True
>>> max(open('data.txt')) # 字符串值最大的行'Testing file IO\n'
>>> min(open('data.txt')) # 用例欢迎补充！'Learning Python, 6E\n'
```

还有一个值得一提的迭代工具，虽然它是本书下一部分的预告：在第 18 章你会学到，函数调用中可以使用一种特殊的形式，把一个集合的值解包成独立的参数，就像它在列表（及其他）字面量中做的那样。你现在大概能猜到：它也接受任何可迭代对象：

```
>>> def f(a, b, c): # 见第四部分
print(a, b, c, sep='&')
>>> f(open('data.txt')) # 也是按行迭代！Testing file IO & Learning Python, 6E & Python 3.12
```

事实上，由于调用中的这种参数解包语法接受可迭代对象，我们还可以用 zip 内建函数来解压（unzip）被压缩的元组：把先前或嵌套的 zip 结果作为另一个 zip 调用的参数（警告：如果你最近打算操作重型机械，最好不要读下面的例子！）：

```
>>> X, Y = (1, 2), (3, 4)
>>> list(zip(X, Y)) # 压缩元组：返回可迭代对象[(1, 3), (2, 4)]
>>> A, B = zip(zip(X, Y)) # 解开 zip，真的！
>>> A, B ((1, 2), (3, 4))
```

我们的迭代工具压轴大戏到此结束。它可能不算完整，但你应该明白要点了。

深度理解

·核心概念：这是本章的"统一场论"——Python 中一切"从左到右扫描"的机制都是迭代协议的消费者：for、推导式、map/filter/zip/enumerate、sorted/sum/any/all/max/min、list/tuple/set/dict 构造器、join、序列赋值、成员测试、切片赋值、extend、星号解包、f(iterable)...学通协议，等于一次性解锁整个标准工具集。

·底层实现：这些工具的 C 实现全部只依赖 tpiter/tpiternext 两个槽位。例如 sorted 内部把迭代器拉进一个临时数组再排序（因此要占内存、不是惰性的）；in 成员测试对无 contains 的对象退化为"逐个 next 比较"；序列赋值 a, b, c = file 编译为反复 next 直到凑够元素；L[1:3] = open(...) 会把迭代结果替换进切片；extend 内部循环 next 并逐个 append（所以是"迭代式"的），而 append 只是把整个对象作为一个元素放进列表。

·为什么这样设计：协议提供了"一次实现、处处可用"的杠杆：本书后面任何实现了协议的用户类，自动获得上面所有工具的支持，无需为每种工具单独适配。这正是 Lutz 所说的"免费的午餐"。

·生活例子：迭代协议像"统一扫码支付"——任何一个店铺（对象）只要挂上同一套收款码（实现协议），所有支付 App（工具）都能直接扫它付款。你不需要为每个 App 单独开通。

·初学者误区：① append 和 extend 混用——append 把可迭代对象当"一个整体"塞进列表，extend 才展开迭

代；②以为 sorted 是惰性的——它必须构造完整列表（与 reversed 不同）；③用 any/all 对文件判断"是否有空行"等想当然的用例——any(file) 对非空文件恒为 True，几乎没意义（书中注释"用例欢迎补充！"就是自嘲）；④不解 zip(zip(...)) 的"解压"戏法——把 zip 结果按行展开成一个个元组参数，外层 zip 再逐列配对，等价于矩阵转置。

·实际问题：真实场景里"一勺烩"的工具组合极其常见——dict(zip(keys, values)) 建映射、','.join(lines) 拼 CSV、max(row) 找最大、f(parts) 动态传参。协议是这些惯用法的共同根基。

代码分析

```
>>> L = [11, 22, 33, 44]
>>> L[1:3] = open('data.txt')    #
>>> L
[11, 'Testing file IO\n', 'Learning Python, 6E\n', 'Python 3.12\n'...]

>>> L = [11]
>>> L.extend(open('data.txt'))    # extend
>>> L
[11, 'Testing file IO\n', 'Learning Python, 6E\n', 'Python 3.12\n']

>>> a, *b = open('data.txt')      # a b
>>> a, b
('Testing file IO\n', ['Learning Python, 6E\n', 'Python 3.12\n'])

>>> X, Y = (1, 2), (3, 4)
>>> A, B = zip(*zip(X, Y))        # * zip
>>> A, B
((1, 2), (3, 4))
```

解释器如何处理：切片赋值 `L[1:3] = iterable` 编译为 `STORESLICE` 操作：先对右侧对象执行 `iter` 取迭代器，然后反复 `next` 直到 `StopIteration`，把取到的元素序列一次性替换进切片位置（内部先收集后替换，所以安全）。`extend` 走 C 循环，逐元素 `listappend`，性能接近推导式。`a, b = open(...)` 的字节码 `UNPACKEX` 会先拉取迭代器，取第一个元素给 `a`，剩余全部收进列表 `b`（`b` 的长度由数据决定）。`zip(zip(X, Y))`：内层 `zip(X, Y)` 产生 `(1,3),(2,4)` 两个元组，把它们展开为两个独立参数传给外层 `zip`，外层 `zip` 再对这两个元组“逐列配对”——取每个元组的第 0 项得 `(1,2)`，第 1 项得 `(3,4)`，于是 `A, B = ((1,2), (3,4))`。这正是二维数组的转置操作，也是“`unzip`”名字的由来。

设计思想：语言层面把“迭代”做成所有数据搬运机制的公共接口（赋值、传参、拼接、构造、成员判定……），让“数据源”与“消费方式”彻底解耦——这是 Python“一致而有限地相互作用”（第 1 章 `fit your brain`）原则在本章最集中的体现。

14.15 其他迭代主题（Other Iteration Topics）

英文原文

As mentioned in this chapter's introduction, there is more coverage of both list comprehensions and iterables in Chapter 20, in conjunction with functions, and

again in Chapter 30 when we study classes. As you'll see later:- User-defined functions can be turned into iterable generator functions, with yield statements.- List comprehensions morph into iterable generator expressions when coded in parentheses.- User-defined classes are made iterable with iter or getitem in operator overloading. In particular, user-defined iterables defined with classes allow arbitrary objects and operations to be used in any of the iteration tools we've met in this chapter. By supporting just a single operation—iteration—objects may be used in a wide variety of contexts and tools.

中文翻译

正如本章引言提到的，第 20 章会在函数的配合下更充分地覆盖列表推导式和可迭代对象，第 30 章研究类时还会再来一次。稍后你会看到：- 用户自定义函数可以用 yield 语句变成可迭代的生成器函数（generator functions）。- 列表推导式写成圆括号形式会蜕变成可迭代的生成器表达式（generator expressions）。- 用户自定义类通过操作符重载（operator overloading）中的 iter 或 getitem 变得可迭代。特别是，用类定义的用户自定义可迭代对象，允许任意对象和操作被用于我们本章见过的任何迭代工具中。只要支持迭代这一个操作，对象就可以在各种各样的语境和工具中使用。

深度理解

·核心概念：本章只是迭代三部曲的"第一部"。后续两条主

线：yield 生成器（第 20 章）与 iter/getitem 类协议（第 30 章）。

·底层实现（预告）：yield 把函数编译成"暂停/恢复"式的生成器对象（每次 next 从上次暂停处继续）；getitem 协议是"旧式迭代"——for 循环会退化为从下标 0 开始反复 obj[0]、obj[1].....直到 IndexError；iter 则是"新式迭代"——for 优先用 iter() 获取迭代器。

·为什么这样设计：把"可迭代性"定义为"只支持一个操作"，就给了对象最大的自由：任何类只需实现一个方法，就能在整个语言的所有迭代工具中畅行无阻——这是最小接口 + 最大化用途的设计哲学。

·生活例子：生成器函数像"按需续期的小说连载"（每次叫它才写下一回）；getitem 像老式电话总机（按号码一个个转接，拨到空号（IndexError）就挂断）；iter 像自助取餐台（直接告诉你怎么自己拿）。

·初学者误区：① 以为"可迭代"需要实现一堆方法——其实核心就一两个；② 把生成器函数和普通函数混淆（调用生成器函数得到的是生成器对象，函数体并不立即执行）；③ 现在就想深挖 yield/getitem——Lutz 特意把它们留到后面，本章只需建立"有后续"的认知。

·实际问题：读完本章，你已经能读懂"任何可迭代对象 + 任何迭代工具"的排列组合代码；接下来的学习路线就是"自己造可迭代对象"。

14.16 章节小结 (Chapter Summary)

英文原文

In this chapter, we explored concepts related to looping in Python. We took our first substantial look at the iteration protocol in Python—a way for nonsequence objects to take part in iteration loops—and at list comprehensions. As we saw, a list comprehension is an expression similar to a for loop that applies another expression to all the items in any iterable object. Along the way, we also saw many of the other built-in iteration tools in Python's arsenal. This wraps up our tour of specific procedural statements and related tools. The next chapter closes out this part of the book by discussing documentation options for Python code. Though a bit of a diversion from the more detailed aspects of coding, documentation is also part of the general syntax model, and it's an important component of well-written programs. In the next chapter, we'll also dig into a set of exercises for this part of the book before we turn our attention to larger structures such as functions. As usual, though, let's first exercise what we've learned here with a quiz.

中文翻译

在本章中，我们探索了与 Python 循环相关的概念。我们第一次实质性地了解了 Python 中的迭代协议——一种让非

序列对象参与迭代循环的方式——以及列表推导式。正如我们所看到的，列表推导式是一个类似于 for 循环的表达式，它把另一个表达式应用到任何可迭代对象中的所有元素上。在此过程中，我们还看到了 Python 武器库中的许多其他内建迭代工具。这结束了对具体过程式语句及相关工具的巡礼。下一章将讨论 Python 代码的文档（documentation）选项，为本书的这一部分收尾。虽然这与编码的细节部分相比算是个小小的岔路，但文档也属于通用语法模型的一部分，并且是良好程序的重要组成部分。在下一章，我们还会深入研究本部分的一组练习，然后转向函数这样更大的结构。不过照例，让我们先用一个小测验来练习一下本章学到的内容。

14.17 测验：检验你的知识 (Test Your Knowledge: Quiz)

英文原文

1. How are for loops and iterable objects related?
2. How are for loops and list comprehensions related?
3. Name four iteration tools in the Python language.
4. What is the best way to read line by line from a text file today?

中文翻译

1. for 循环和可迭代对象有什么关系？
2. for 循环和列表推导式有什么关系？
3. 说出 Python 语言中的四种迭代工具。
4. 如今逐行读取文本文件的最佳方式是什么？

14.18 测验答案 (Test Your Knowledge: Answers)

英文原文

1. The for loop normally uses the iteration protocol to step through items in the iterable object across which it is iterating. It first fetches an iterator from the iterable by calling the iterable's `iter`, and then calls this iterator object's `next` method on each iteration to advance and catches its `StopIteration` exception to determine when to stop looping. Any object that supports this model works in a for loop and in all other iteration tools. The protocol's methods can also be run manually with built-ins `iter` and `next`. For some objects that are their own iterator, the initial `iter` call is extraneous but harmless. 2. Both are iteration tools. List comprehensions are a concise and often efficient way to perform a common for loop task: collecting the results of applying an expression to all items in an iterable object. It's always possible to translate a list comprehension to a for loop, and part of the list comprehension expression looks like the header of a for loop syntactically. The for loop, however, can be used in additional looping roles that comprehensions do not address. 3. Iteration tools in Python include the for loop; list comprehensions; the `map` built-in function; the

in membership test expression; and the built-in functions sorted, sum, any, and all. This category also includes the list and tuple built-ins, string join methods, and sequence assignments (starred or not), all of which use the iteration protocol (see answer #1) to step across iterable objects one item at a time. 4. The "best" way to read lines from a text file today is to not read it explicitly at all: instead, open the file within an iteration tool such as a for loop or list comprehension, and let the iteration tool automatically scan one line at a time by running the file's next handler method on each iteration. This approach is generally best in terms of coding simplicity, memory space, and possibly execution speed requirements.

中文翻译

1. for 循环通常使用迭代协议来逐步遍历它所迭代的可迭代对象中的元素。它首先通过调用可迭代对象的 iter 从可迭代对象获取一个迭代器，然后在每次迭代时调用该迭代器对象的 next 方法前进，并捕获其 StopIteration 异常来判断何时停止循环。任何支持该模型的对象都能用于 for 循环以及所有其他迭代工具。协议的这些方法也可以用内建函数 iter 和 next 手动运行。对某些"自己是自己的迭代器"的对象来说，最初的 iter 调用是多余的，但无害。2. 两者都是迭代工具。列表推导式是执行一个常见 for 循环任务的简洁且通常高效的方式：收集"把表达式应用到可迭代对象中所有元素"的结果。列表推导式总能翻译成 for 循环，而且列表推导式表达式的一部分在语法上看起来就像 for 循环头。不

过，for 循环还能用于推导式所不涉及的额外循环角色。3. Python 中的迭代工具包括：for 循环；列表推导式；map 内建函数；in 成员测试表达式；以及内建函数 sorted、sum、any 和 all。这一类还包括 list 和 tuple 内建函数、字符串 join 方法、以及序列赋值（带不带星号都算），它们全部使用迭代协议（见答案 1）一次一个元素地遍历可迭代对象。4. 如今读取文本文件行的"最佳"方式是根本不去显式地读它：而是在 for 循环或列表推导式这样的迭代工具中打开文件，让迭代工具在每次迭代时通过运行文件的 next 处理方法自动逐行扫描。就编码简洁性、内存占用以及可能还有执行速度需求而言，这种方法通常是最佳的。

本章总结

技术拓展 (Technical Expansion)

- 实际项目中的应用场景
- 流式大文件处理：日志分析、超大 CSV/JSONL 解析——永远用 `for line in open(f)` 或生成器表达式逐行处理，绝不 `readlines()` 全量载入。这是本章"文件迭代器 vs `readlines`"对比的直接落地。
- 数据管道 (pipeline)：`list(zip(keys, values))` 建字典、`dict(zip(ids, records))` 做索引映射、`filter(None, data)` 去假值、`','.join(str(x) for x in nums)` 拼字符串——把本章的迭代工具按需组合成一条条一行式管道。
- 批量变换与清洗：`[row.rstrip().split(',') for row in open(f)]`

n(csv) if row.strip()] 这类"逐行读取 + 变换 + 过滤"的推导式，是数据清洗的第一选择；数据量大时把方括号换成圆括号（生成器表达式，见第 20 章）即可省内存。

· "解包即用"技巧：a, rest = iterable 取首元素与其余部分；zip(matrix) 做矩阵转置；f(parts) 把行数据动态传给函数——这些是算法题与日常脚本的高频手法。

· 惰性求值避免延迟：当"只要前几个结果"时（如 next(itertools.islice(data, 5))），惰性迭代器让计算成本随消费量增长，而非一次付清。

· 与其他语言（Java/C++）的区别

维度	Python (迭代协议)	Java (Iterable /Iterator)	C++ (STL 迭代器)
协议本质	iter/next + StopIteration 异常	Iterator.hasNext() + next() (hasNext 显式询问)	泛型指针算术 (begin/end 区间)
终止信号	异常 (隐式、自动捕获)	方法返回值 (显式检查)	指针与 end() 比较
迭代工具	for/推导式/map/zip/sorted/sum...全部语言	仅增强 for、Stream (Java 8+)	算法库 <algorithm> (foreach 等)
惰性程度	语言级设计 (range/zip/generator 默认惰性)	Stream 支持惰性，集合本身非惰性	istreamiterator 惰性，容器非惰性
实现成本	任何对象实现两个方法即可被所有工具消费	需实现接口，配合 Stream 需额外学习	迭代器类别 (输入/双向/随机访问) 体系复杂

一句话概括：Python 用"异常 + 协议"把迭代做成了语言的无处不在的默认机制；Java 把它做成显式接口；C++ 把它做成与容器绑定的泛型抽象。

· Python 发展历史背景

- 1990 年 Python 诞生时只有序列 (list、tuple) 可被 for 遍历, 靠下标索引完成。
- 1998 年 Python 2.2 引入迭代协议 (PEP 234) : iter/next/StopIteration, 同时引入 iter()/next() 内建函数与生成器 (PEP 255) , "非序列对象参与循环"成为可能。文件、字典、zip 等从那时起获得迭代器。
- 2001 年 Python 2.0 引入列表推导式 (灵感来自 Haskell 与数学集合论描述法) ; 随后 Python 2.4 加入生成器表达式 (PEP 289) , Python 2.7/3.0 加入集合与字典推导式 (PEP 274) 。
- 2008 年 Python 3 把 next() 方法统一命名为 next (PEP 3114) , 并让推导式拥有独立作用域 (不再泄漏循环变量) 。
- Python 3.8 的海象运算符 := (PEP 572) 可以配合 next(l, None) 写出极简的消费循环——本书 14.7 节演示过其用法与陷阱。
- 性能史: 迭代器逐行读文件之所以比 readlines/while 快, 是因为它在 C 层工作; 但 Lutz 的"速度声明"提醒我们——这些相对性能结论会随 CPython 版本演进而变化。
- 高级开发者需要掌握的相关知识
- 惰性求值的成本与边界: 惰性省内存, 但不是免费——生成器无法随机访问、无法 len、调试困难; "省内存 vs 需要多遍扫描"的权衡要按场景判断。
- itertools 标准库: chain、product、islice、groupby、cycle、repeat——它们都是"迭代工具的组合积木",

与本章协议一脉相承，是下一步最值得学的模块。

·自定义迭代器的正确姿势（第 20/30 章预告）：生成器函数（yield）、生成器表达式、iter/next 类实现、旧式 getitem 协议；理解"自身即迭代器"与"返回新迭代器"两种模式的适用性。

·性能测量（第 21 章）：用 timeit 科学地比较 for 循环与推导式，而不是凭印象断言"推导式快 2 倍"。

·协议在框架中的身影：Django 的 QuerySet、Pandas 的 DataFrame 行迭代、asyncio 异步迭代协议（aiter/next）——都是同一思想的延展。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5)。迭代协议是 Python 的"底层操作系统"之一，比任何单一语法都更根本；不掌握它，第 20 章生成器、第 30 章类协议、乃至整个标准库的一半用法都会难以理解。

·应该掌握到什么程度：

·必须脱口而出：迭代协议四要素（iter → iterator、next → 结果、StopIteration → 终止）；iter(f) is f（文件/zip/enumerate 是自身迭代器）vs iter(L) is L 为 False（列表可多次扫描）。

·能默写：手动迭代等价代码（l = iter(L); while True: try: x = next(l) except StopIteration: break）。

·能熟练转换：推导式 for 循环双向翻译（append + 缩进规则）；理解 if 过滤、嵌套 for 的顺序语义。

·能流利使用：for + 文件迭代器逐行读文件；zip/enumerate/map/filter/sorted 对任意可迭代对象的基本组合；next(l, default) 与哨兵 iter(func, sentinel) 两个少见但有用的模式。

·记住边界：单遍对象不能重复扫描；append 不迭代、extend 迭代；sum/any/all/max/min 消费迭代器返回单一结果。

·后续应该学习哪些相关内容：

·立即衔接：第 15 章（文档——语法模型的收尾）、第 18 章（函数参数与解包）、第 19 章（函数式编程：filter/reduce/函数式工具）、第 21 章（timeit 性能测量）。

·核心进阶：第 20 章（生成器函数、yield、生成器表达式——本章所有工具的函数版续篇）、第 30 章（用类实现 iter/getitem——本章协议的类版续篇）。

·实战演练：用 for line in open(f) 写一个逐行统计工具；用嵌套推导式生成九九乘法表；用 zip(matrix) 实现转置；把第 13 章的每个 for 循环改写成推导式，再比较可读性与（用 timeit 测得的）性能。

·延伸阅读：Python 官方文档 "The Python Tutorial: Iterator Types"、PEP 234、itertools 模块文档。

本章完。下一章：Documentation（文档），之后将进入函数的世界。